Reproducing and Improving TypiClust

Lucas Lobo - k23075501

1 Introduction

Deep learning requires large labelled datasets, yet annotation is costly. Active Learning (AL) mitigates this by iteratively selecting informative unlabelled examples, but traditional uncertainty-based methods rely on a trained model, creating a cold start problem. Hacohen et al. [1] identified that the optimal AL strategy depends on budget size: typical examples should be queried first, with uncertainty-based methods taking over once sufficient labels exist. They proposed TypiClust, which selects typical points from K-Means clusters in a self-supervised representation space. However, typicality alone can produce redundant selections. We reproduce TypiClust on CIFAR-10 across all three evaluation frameworks with 8 AL baselines, and propose a hybrid modification incorporating diversity-aware selection.

2 Methodology

2.1 TypiClust Pipeline

Our implementation follows the TPC_{RP} variant for initial pool selection ($L_0 = \emptyset$). Following the paper (Appendix F.1), we train a SimCLR [2] encoder (ResNet-18, 500 epochs) from scratch on CIFAR-10, using SGD (lr=0.4, momentum 0.9, cosine schedule, batch 512, weight decay 10^{-4}) and L2-normalised 512-d embeddings as the representation space. The resulting linear probe achieves 89.6% on the full training set. The algorithm proceeds as: (1) cluster embeddings with K-Means ($K = B$), (2) from each cluster, select the point with highest typicality:

$$\text{Typicality}(x) = \left(\frac{1}{k} \sum_{x_i \in \text{KNN}(x)} \|x - x_i\|_2 \right)^{-1} \tag{1}$$

where $k = 20$ neighbours. We evaluate across all three frameworks: (i) fully supervised (ResNet-18 trained from scratch), (ii) SS embedding (linear probe on frozen embeddings), and (iii) semi-supervised (pseudo-labelling), comparing against 8 baselines: Random, Uncertainty, Margin, Entropy, CoreSet [4], BADGE, BALD, and DBAL.

2.2 Proposed Modification

The paper's theoretical analysis (Section 2) shows that in the low-budget regime, selecting from high-density regions (R_1) is optimal, requiring both *max density* (typicality) and *diversity*. TypiClust enforces diversity via clustering, but purely typical selections can still be spatially redundant within clusters. To address this, we propose:

Hybrid Score Formula. We replace the pure typicality criterion with a hybrid score:

$$\text{score} = \alpha \cdot \hat{t} + (1 - \alpha) \cdot \hat{d} \tag{2}$$

where $\hat{t}$ is the normalised typicality and $\hat{d}$ is the normalised minimum distance to already-selected points, both scaled to $[0, 1]$ within each cluster. Selection proceeds sequentially: the most typical point overall is chosen first, and each subsequent point is scored considering distance from already-selected points.

Dimensionality Reduction. High-dimensional embeddings (512-d) can suffer from the curse of dimensionality [5], where distance metrics become less meaningful. We apply PCA and UMAP [3] to reduce dimensionality before clustering. PCA preserves global variance structure while UMAP captures nonlinear manifold geometry. Therefore, we sweep both to find the optimal configuration.

Alpha Tuning. α controls the typicality-diversity trade-off: $\alpha = 1$ recovers original TypiClust, $\alpha = 0$ selects purely for distance. We sweep α across $[0, 1]$.

3 Results

3.1 Paper Results and Reproduction

Table 1 compares the paper's results to our reproduction on CIFAR-10 ($B = 10$). In the SS embedding framework, our TypiClust achieves +9.8% over random, closely matching the paper's ∼+10%. Our slightly higher absolute values (42% vs ∼38% random) likely reflect minor differences in SimCLR training code vs the paper's Van Gansbeke et al. implementation. The paper additionally reports 93.2% in the semi-supervised setting (FlexMatch), which we approximate with pseudo-labelling (see Limitations).

	Random	TPC_{RP}	Δ	Source
Paper: Supervised	∼18%	∼28%	+10%	Fig. 4
Paper: SS Embed.	∼38%	∼48%	+10%	Fig. 5
Paper: Semi-sup.	53.8%	93.2%	+39.4%	Fig. 6
Ours: Supervised	15.4%	12.9%	−2.5%	fw1
Ours: SS Embed.	42.1%	51.8%	**+9.8%**	fw2
Ours: Semi-sup.	12.4%	15.8%	+3.4%	fw3

Table 1: Paper vs our reproduction ($B = 10$). SS embedding closely matches. The fully supervised gap at $B = 10$ is due to 9/10 class coverage (see Limitations), semi-supervised uses pseudo-labelling vs FlexMatch.

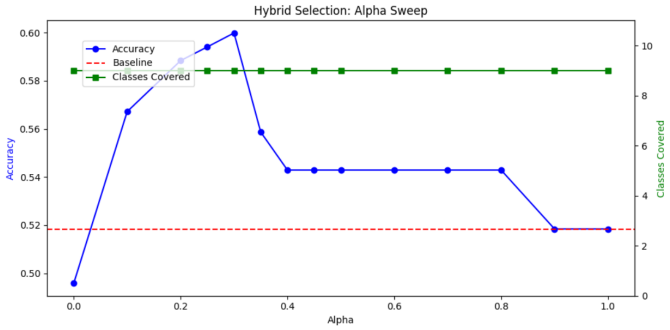


Figure 1: Hybrid selection on CIFAR-10 ($B = 10$): accuracy and class coverage vs. α. Dashed line shows baseline TypiClust.

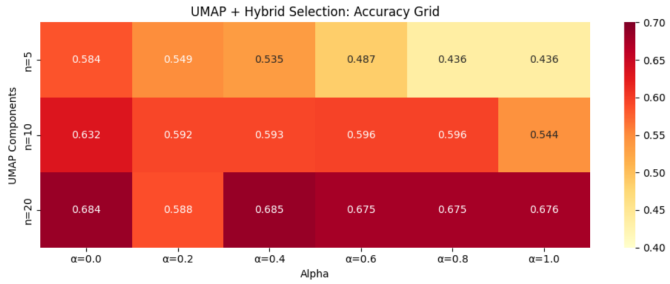


Figure 2: UMAP + hybrid accuracy grid on CIFAR-10 ($B = 10$) across UMAP dimensions and α.

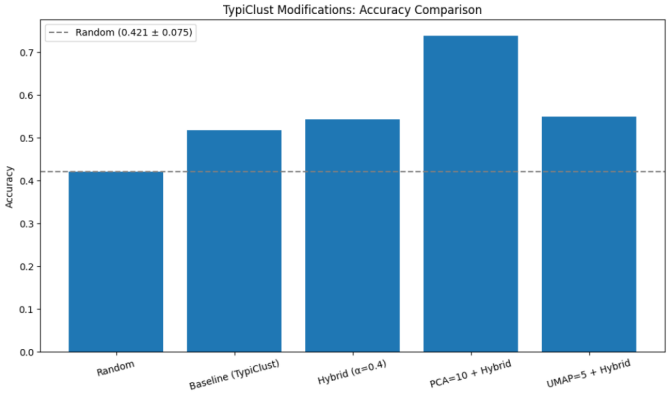


Figure 3: Accuracy comparison across all selection methods on CIFAR-10 ($B = 10$). Dashed line shows random baseline mean.

Method	Accuracy	Classes
Random (30 seeds)	$(42.07 \pm 7.45)\%$	–
TypiClust (baseline)	$(51.84 \pm 0.00)\%$	9
Hybrid ($\alpha = 0.4$)	$(54.29 \pm 0.00)\%$	9
UMAP-20 + Hybrid	$(68.51 \pm 3.62)\%$	–
PCA-10 + Hybrid	$(\mathbf{73.91 \pm 0.00})\%$	10

Table 2: Selection strategies on CIFAR-10 ($B = 10$).

3.2 Our Results

In the SS embedding framework, multi-round experiments across 5 AL rounds with all 8 baselines confirm that TypiClust dominates (51.9% at $B = 10$, rising to 81.6% at $B = 40$), while Margin overtakes at $B = 50$ (82.4% vs 80.8%), reproducing the phase transition (Appendix, Figure 6). In the fully supervised framework, all methods cluster between 13–19% at $B = 10$ with high variance, consistent with the paper's finding that training a network on 10 examples is inherently noisy. In semi-supervised, TypiClust's advantage is amplified at $B = 20$ (67.2% vs random 28.4%), confirming the paper's key result.

The alpha sweep (Figure 1) shows that hybrid selection ($\alpha = 0.4$) raises accuracy to 54.29%. The best result is PCA ($n = 10$, $\alpha = 0.55$): 73.91% with full class coverage (10/10), a +22.07% improvement over baseline (Figure 3). PCA outperforms UMAP because linear projection preserves the well-structured embedding geometry. UMAP's nonlinear distortion at low dimensions discards discriminative information, though it improves at $n = 20$ (68.5%, Figure 2).

We also evaluate over-clustering ($K > B$) with both PCA and UMAP. Over-clustering consistently degrades performance, validating the paper's $K = B$ design.

4 Statistical Analysis

The random baseline 95% CI is $[39.4\%, 44.7\%]$, TypiClust at 51.84% falls well above this, confirming a significant reproduction. For the modification, we compare each method (10 seeds) against the random baseline (30 seeds) using two-sample t-tests. All achieve significance: baseline ($t = 4.04$, $p < 0.001$, $d = 1.85$), hybrid ($t = 5.05$, $p < 0.001$, $d = 2.32$), PCA + hybrid ($t = 13.17$, $p < 0.001$, $d = 6.04$), UMAP + hybrid ($t = 9.78$, $p < 0.001$, $d = 4.19$). With Bonferroni correction ($\alpha_{\text{adj}} = 0.0125$), all remain significant.

5 Conclusion

We reproduce TypiClust on CIFAR-10 across three evaluation frameworks with 8 AL baselines. In the SS embedding framework, TypiClust outperforms all baselines at low budgets (+9.77% over random at $B = 10$) and dominates through $B = 40$, while a phase transition occurs where Margin surpasses it at $B = 50$. Our hybrid modification combining typicality with diversity-aware selection and PCA dimensionality reduction improves accuracy from 51.84% to 73.91% (+22.07%), achieving full class coverage.

Limitations. Our SimCLR merges two classes (dog/horse) into one cluster, limiting TypiClust to 9/10 coverage. Causing issues for fully supervised but manageable for SS embedding. Fully supervised uses 200 epochs/5 reps (vs paper's Munjal framework). Semi-supervised uses pseudo-labelling vs FlexMatch (infeasible: WideResNet-28, 400k iterations). **Code:** `https://github.com/LucasPRLobo/ml-cw2`

References

[1] G. Hacohen, A. Dekel, and D. Weinshall. Active learning on a budget: Opposite strategies suit high and low budgets. *ICML*, PMLR 162, 2022.

[2] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. *ICML*, pp. 1597–1607, 2020.

[3] L. McInnes, J. Healy, and J. Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv:1802.03426*, 2018.

[4] O. Sener and S. Savarese. Active learning for convolutional neural networks: A core-set approach. *ICLR*, 2018.

[5] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. *ICDT*, pp. 420–434, 2001.

Additional Plots

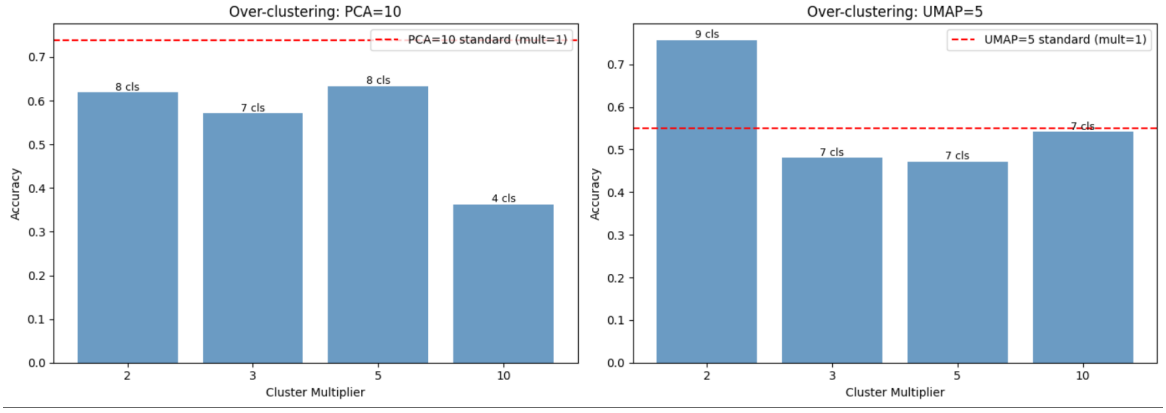


Figure 4: Over-clustering ablation for PCA-10 and UMAP-5. Dashed lines: standard $K = B$ result. Over-clustering consistently degrades performance.

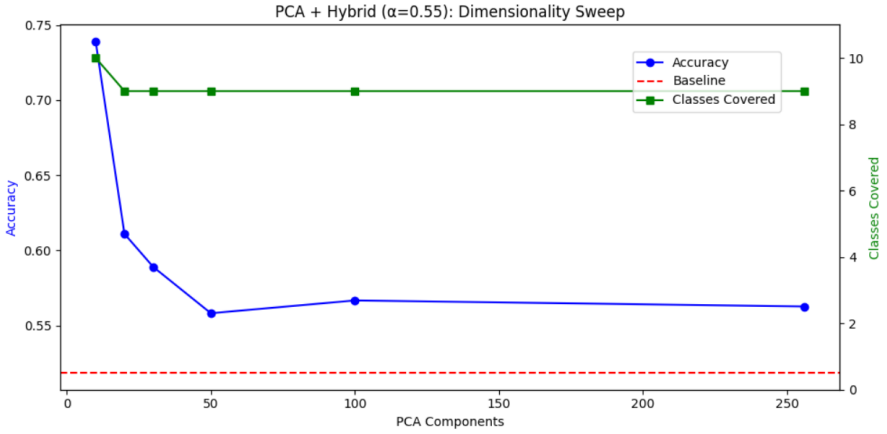


Figure 5: PCA + hybrid ($\alpha = 0.55$): accuracy and class coverage across PCA dimensions.

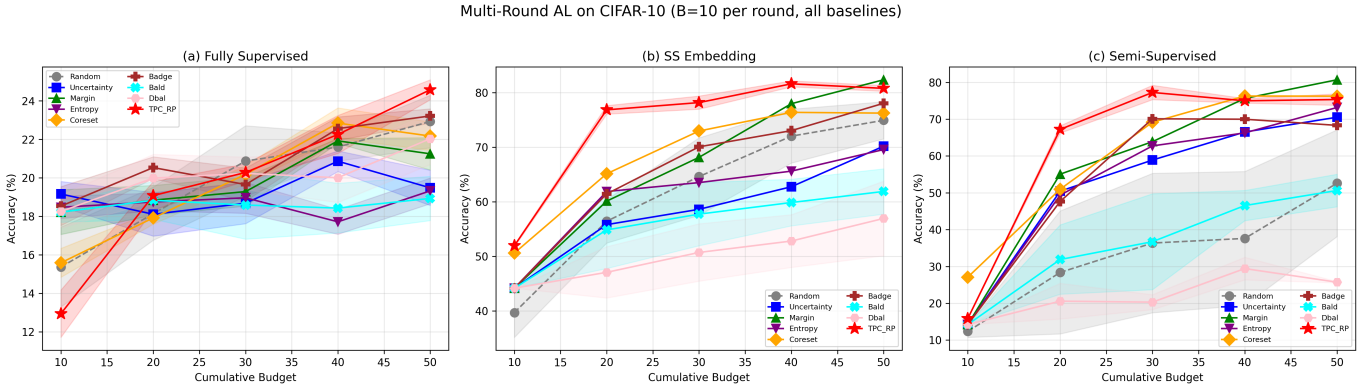


Figure 6: Multi-round AL comparison on CIFAR-10 across all three evaluation frameworks. (a) Fully supervised: all methods perform similarly (13–25%). TypiClust underperforms at $B = 10$ due to 9/10 class coverage but recovers at $B = 50$. (b) SS embedding: TypiClust dominates from $B = 10$ through $B = 40$. Margin overtakes at $B = 50$, reproducing the phase transition from [1]. (c) Semi-supervised (pseudo-labelling): TypiClust's advantage is amplified at $B = 20$ (67.2% vs random 28.4%). Margin overtakes at $B = 50$ (80.7%).

original_algorithm

March 24, 2026

1 Original TypiClust Algorithm

Reproduction of TPC_RP (SimCLR + K-Means + Typicality Selection) on CIFAR-10 with B=10.

Note: The SimCLR encoder (ResNet-18, 200 epochs) is trained from scratch in a separate notebook (`SimCLR_implementation.ipynb`). The training is split out so the algorithm can be run without retraining each time. The saved weights are loaded below.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
 ↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            h = torch.nn.functional.normalize(h, dim=1)  # L2 normalize (paper␣
 ↪F.1)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_500ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. Typicality Computation

[5]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [1.0799, 18.4421]
```

1.4 4. Clustering (K-Means, K=B)

[6]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_


budget = 10
cluster_labels = cluster_standard(embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [5314 3763 4912 5582 4727 2818 4972 4982 6392 6538]
```

1.5 5. Selection (Max Typicality)

[7]:
```
def select_max_typicality(typicality, cluster_labels, budget):
    """Select the most typical point from each cluster."""
    selected = []
    for cluster in range(budget):
        cluster_idx = np.where(cluster_labels == cluster)[0]
        best = cluster_idx[np.argmax(typicality[cluster_idx])]
        selected.append(best)
    return np.array(selected)


selected_indices = select_max_typicality(typicality, cluster_labels, budget)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
```

```
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [38224 29916 18943 38604 42923 39686 30062  7617 48591 37173]
Selected labels: [0 7 9 7 6 2 1 8 4 3]
Classes covered: 9
```

1.6 6. Evaluation (Linear Probe)

[8]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, embeddings, labels,␣
↪test_embeddings, test_labels)
print(f"\n=== Original TypiClust Results ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Original TypiClust Results ===
Accuracy: 0.5184
Classes covered: 9/10
```

1.7 7. Random Baseline Comparison

[9]:
```python
# Random baseline (30 seeds)
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[idx], labels[idx])
    preds = clf.predict(test_embeddings)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)
print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"TypiClust: {results['accuracy']:.4f}")
print(f"Improvement: +{results['accuracy'] - random_accs.mean():.4f}")
```

```
Random Baseline: 0.4207 +/- 0.0745
TypiClust: 0.5184
Improvement: +0.0977
```

modified_algorithm

March 24, 2026

1 Modified TypiClust Algorithm

Best configuration: PCA (n=10) + Hybrid Selection (alpha=0.55) on CIFAR-10 with B=10.

1.1 1. Setup & Data Loading

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
```

[2]:
```
# Load CIFAR-10
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)
```

1.2 2. SimCLR Encoder (Representation Learning)

[3]:
```
class ResNet18Encoder(nn.Module):
    """SimCLR encoder with ResNet-18 backbone adapted for CIFAR-10 (32x32)."""
    def __init__(self):
```

```
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            h = torch.nn.functional.normalize(h, dim=1)  # L2 normalize (paper␣
↪F.1)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

[4]:
```
# Load pretrained SimCLR encoder and extract embeddings
encoder = load_encoder('./simclr_encoder_500ep.pth')
embeddings, labels = extract_embeddings(encoder, trainloader)
test_embeddings, test_labels = extract_embeddings(encoder, testloader)
print(f"Train embeddings: {embeddings.shape}")
print(f"Test embeddings: {test_embeddings.shape}")
```

```
Train embeddings: (50000, 512)
Test embeddings: (10000, 512)
```

1.3 3. PCA Dimensionality Reduction (Modification 1)

[5]:
```
def preprocess_pca(embeddings, n_components=10):
    """Reduce embedding dimensionality with PCA before clustering."""
    pca = PCA(n_components=n_components)
    transformed = pca.fit_transform(embeddings)
    return transformed, pca


# Apply PCA (n=10) to train and test embeddings
proc_embeddings, pca_model = preprocess_pca(embeddings, n_components=10)
proc_test = pca_model.transform(test_embeddings)
print(f"PCA train embeddings: {proc_embeddings.shape}")
print(f"PCA test embeddings: {proc_test.shape}")
print(f"Explained variance ratio: {pca_model.explained_variance_ratio_.sum():.
 ↪4f}")
```

```
PCA train embeddings: (50000, 10)
PCA test embeddings: (10000, 10)
Explained variance ratio: 0.4153
```

1.4 4. Typicality Computation

[6]:
```
def compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1


typicality = compute_typicality(proc_embeddings, k=20)
print(f"Typicality shape: {typicality.shape}")
print(f"Typicality range: [{typicality.min():.4f}, {typicality.max():.4f}]")
```

```
Typicality shape: (50000,)
Typicality range: [2.6835, 86.8538]
```

1.5 5. Clustering (K-Means, K=B)

[7]:
```
def cluster_standard(embeddings, budget, random_state=42):
    """K-Means clustering with K = budget."""
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_
```

```
budget = 10
cluster_labels = cluster_standard(proc_embeddings, budget)
print(f"Cluster sizes: {np.bincount(cluster_labels)}")
```

```
Cluster sizes: [3803 4182 6327 4949 4946 4638 6124 5681 6018 3332]
```

1.6 6. Hybrid Selection (Modification 2)

[8]:
```
def select_hybrid(embeddings, typicality, cluster_labels, budget, alpha=0.55):
    """
    Hybrid selection: score = alpha * typicality + (1-alpha) * min_distance.
    Selects sequentially, balancing representativeness with spatial diversity.
    """
    selected = []

    # Order clusters by max typicality (most typical cluster first)
    cluster_order = []
    for c in range(budget):
        cluster_idx = np.where(cluster_labels == c)[0]
        cluster_order.append((c, np.max(typicality[cluster_idx])))
    cluster_order.sort(key=lambda x: x[1], reverse=True)

    # First point: most typical overall
    first_cluster = cluster_order[0][0]
    cluster_idx = np.where(cluster_labels == first_cluster)[0]
    best = cluster_idx[np.argmax(typicality[cluster_idx])]
    selected.append(best)

    # Subsequent points: hybrid score within each cluster
    for cluster in cluster_order[1:]:
        cluster_idx = np.where(cluster_labels == cluster[0])[0]
        candidate_embeddings = embeddings[cluster_idx]

        # Compute min distance to already-selected points
        dist = np.linalg.norm(
            candidate_embeddings[:, None] - embeddings[selected], axis=2
        )
        min_distance = dist.min(axis=1)

        # Normalise both terms to [0, 1] within this cluster
        norm_distance = (min_distance - min_distance.min()) / (
            min_distance.max() - min_distance.min() + 1e-8
        )
        norm_typicality = (typicality[cluster_idx] - typicality[cluster_idx].
↪min()) / (
            typicality[cluster_idx].max() - typicality[cluster_idx].min() + 1e-8
        )
```

```
        # Hybrid score
        score = alpha * norm_typicality + (1 - alpha) * norm_distance
        best = cluster_idx[np.argmax(score)]
        selected.append(best)

    return np.array(selected)


selected_indices = select_hybrid(proc_embeddings, typicality, cluster_labels,␣
 ↪budget, alpha=0.55)
print(f"Selected indices: {selected_indices}")
print(f"Selected labels: {labels[selected_indices]}")
print(f"Classes covered: {len(np.unique(labels[selected_indices]))}")
```

```
Selected indices: [33063  3766 23733 10212 28528 36746 29705 30391 34326 15152]
Selected labels: [1 9 5 0 7 8 3 2 6 4]
Classes covered: 10
```

1.7 7. Evaluation (Linear Probe)

[9]:
```
def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
 ↪test_labels):
    """Train a linear probe on selected examples and evaluate on test set."""
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(sel_emb, sel_lab)
    preds = clf.predict(test_embeddings)
    accuracy = accuracy_score(test_labels, preds)
    n_classes = len(np.unique(sel_lab))
    return {'accuracy': accuracy, 'n_classes_covered': n_classes}


results = evaluate_selection(selected_indices, proc_embeddings, labels,␣
 ↪proc_test, test_labels)
print(f"\n=== Modified TypiClust Results (PCA-10 + Hybrid alpha=0.55) ===")
print(f"Accuracy: {results['accuracy']:.4f}")
print(f"Classes covered: {results['n_classes_covered']}/10")
```

```
=== Modified TypiClust Results (PCA-10 + Hybrid alpha=0.55) ===
Accuracy: 0.7391
Classes covered: 10/10
```

1.8 8. Comparison with Baseline

[10]:
```
# Random baseline (30 seeds) on PCA-reduced embeddings
random_accs = []
for seed in range(30):
    rng = np.random.RandomState(seed)
    idx = rng.choice(len(proc_embeddings), size=budget, replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(proc_embeddings[idx], labels[idx])
    preds = clf.predict(proc_test)
    random_accs.append(accuracy_score(test_labels, preds))

random_accs = np.array(random_accs)

# Original TypiClust baseline (no PCA, no hybrid) on raw embeddings
from sklearn.cluster import KMeans as KM
km = KM(n_clusters=budget, random_state=42, n_init=10)
km.fit(embeddings)
baseline_typ = compute_typicality(embeddings, k=20)
baseline_sel = []
for c in range(budget):
    cidx = np.where(km.labels_ == c)[0]
    baseline_sel.append(cidx[np.argmax(baseline_typ[cidx])])
baseline_sel = np.array(baseline_sel)
clf = LogisticRegression(max_iter=1000, C=100)
clf.fit(embeddings[baseline_sel], labels[baseline_sel])
baseline_acc = accuracy_score(test_labels, clf.predict(test_embeddings))

print(f"Random Baseline: {random_accs.mean():.4f} +/- {random_accs.std():.4f}")
print(f"Original TypiClust: {baseline_acc:.4f}")
print(f"Modified (PCA-10 + Hybrid): {results['accuracy']:.4f}")
print(f"Improvement over baseline: +{results['accuracy'] - baseline_acc:.4f}")
```

```
Random Baseline: 0.4832 +/- 0.0731
Original TypiClust: 0.5184
Modified (PCA-10 + Hybrid): 0.7391
Improvement over baseline: +0.2207
```

SimCLR_implementation

March 24, 2026

1 SimCLR Implementation

Training hyperparameters follow the paper (Appendix F.1, Van Gansbeke et al. 2020): - **Architecture:** ResNet-18 encoder + MLP projection head (512 → 128) - **Optimizer:** SGD, 0.9 momentum, weight decay 0.0001 - **Learning rate:** 0.4, cosine scheduler - **Batch size:** 512 - **Epochs:** 500 - **Augmentations:** Random resized crop, horizontal flip, color jitter, random grayscale - **Loss:** NT-Xent (=0.5) - **Embedding:** L2-normalized 512-d penultimate layer

[11]:
```
import torch
import torchvision.transforms as transforms
import torch.nn as nn
import torchvision
import torchvision.models as models

from torch.utils.data import DataLoader
from tqdm import tqdm
```

1.0.1 Augmentation Transform

[12]:
```
class SimCLRTransform:
    def __init__(self):
        self.transform = transforms.Compose([
            transforms.RandomResizedCrop(size=32),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.
 ↪1)], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
        ])

    def __call__(self, x):
        return self.transform(x), self.transform(x)
```

1.0.2 ResNet-18 Encoder

[13]:
```
class ResNet18Encoder(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18Encoder, self).__init__()
        self.resnet = models.resnet18(pretrained=False)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()  # Remove maxpool layer for CIFAR-10
        self.resnet.fc = nn.Identity()  # Remove fully connected layer for␣
↪feature extraction

    def forward(self, x):
        return self.resnet(x)
```

1.0.3 Projection Head

[14]:
```
class ProjectionHead(nn.Module):
    def __init__(self, input_dim=512, hidden_dim=128, output_dim=128):
        super(ProjectionHead, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

1.0.4 NT_Xent Loss

[15]:
```
def nt_xent_loss(z_i, z_j, temperature=0.5):
    batch_size = z_i.size(0)
    z = torch.cat([z_i, z_j], dim=0)  # Concatenate positive pairs

    z = nn.functional.normalize(z, dim=1)  # L2 Norm
    similarity_matrix = torch.matmul(z, z.T)/temperature  # Compute similarity␣
↪matrix

    mask = torch.eye(2 * batch_size, dtype=torch.bool).to(z.device)
    similarity_matrix = similarity_matrix[~mask].view(2 * batch_size, -1)  #␣
↪Remove self-similarity

    targets = torch.cat([
        torch.arange(batch_size -1, 2*batch_size -1),
        torch.arange(0, batch_size)
```

```
    ]).to(z.device)

    loss = nn.CrossEntropyLoss()(similarity_matrix, targets)
    return loss
```

1.0.5 Import Dataset

[ ]:
```
simclr_trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                               download=True,␣
 ↪transform=SimCLRTransform())
simclr_loader = DataLoader(simclr_trainset, batch_size=512, shuffle=True,␣
 ↪num_workers=2, drop_last=True)
```

1.0.6 Training Loop Setup

[ ]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Initialize models
encoder = ResNet18Encoder().to(device)
projection_head = ProjectionHead().to(device)

# Optimizer - matches paper: SGD, 0.9 momentum, lr=0.4, cosine schedule, weight␣
 ↪decay 0.0001
num_epochs = 500
optimizer = torch.optim.SGD(
    list(encoder.parameters()) + list(projection_head.parameters()),
    lr=0.4, momentum=0.9, weight_decay=0.0001
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,␣
 ↪T_max=num_epochs)

print(f'Training for {num_epochs} epochs')
print(f'Optimizer: SGD (lr=0.4, momentum=0.9, weight_decay=0.0001)')
print(f'Scheduler: CosineAnnealingLR')
```

1.0.7 Training Loop

[ ]:
```
for epoch in range(num_epochs):
    running_loss = 0.0
    loop = tqdm(simclr_loader, desc=f'Epoch [{epoch+1}/{num_epochs}]',␣
 ↪leave=False)
    for (view1, view2), _ in loop:
        view1, view2 = view1.to(device), view2.to(device)

        h_i = encoder(view1)
        h_j = encoder(view2)
```

```
        z_i = projection_head(h_i)
        z_j = projection_head(h_j)

        loss = nt_xent_loss(z_i, z_j)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        loop.set_postfix(loss=loss.item())

    scheduler.step()
    avg_loss = running_loss / len(simclr_loader)
    if (epoch + 1) % 50 == 0 or epoch == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Avg Loss: {avg_loss:.4f}, LR:␣
↪{scheduler.get_last_lr()[0]:.6f}')

# Save encoder weights
torch.save(encoder.state_dict(), 'simclr_encoder_500ep.pth')
print('Training complete. Encoder saved to simclr_encoder_500ep.pth')
```

SimCLR_implementation

March 24, 2026

1 SimCLR Implementation

Training hyperparameters follow the paper (Appendix F.1, Van Gansbeke et al. 2020): - **Architecture:** ResNet-18 encoder + MLP projection head (512 → 128) - **Optimizer:** SGD, 0.9 momentum, weight decay 0.0001 - **Learning rate:** 0.4, cosine scheduler - **Batch size:** 512 - **Epochs:** 500 - **Augmentations:** Random resized crop, horizontal flip, color jitter, random grayscale - **Loss:** NT-Xent (=0.5) - **Embedding:** L2-normalized 512-d penultimate layer

[11]:
```
import torch
import torchvision.transforms as transforms
import torch.nn as nn
import torchvision
import torchvision.models as models

from torch.utils.data import DataLoader
from tqdm import tqdm
```

1.0.1 Augmentation Transform

[12]:
```
class SimCLRTransform:
    def __init__(self):
        self.transform = transforms.Compose([
            transforms.RandomResizedCrop(size=32),
            transforms.RandomHorizontalFlip(),
            transforms.RandomApply([transforms.ColorJitter(0.4, 0.4, 0.4, 0.
 ↪1)], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
            transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
        ])

    def __call__(self, x):
        return self.transform(x), self.transform(x)
```

1.0.2 ResNet-18 Encoder

[13]:
```
class ResNet18Encoder(nn.Module):
    def __init__(self, num_classes=10):
        super(ResNet18Encoder, self).__init__()
        self.resnet = models.resnet18(pretrained=False)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()  # Remove maxpool layer for CIFAR-10
        self.resnet.fc = nn.Identity()  # Remove fully connected layer for␣
↪feature extraction

    def forward(self, x):
        return self.resnet(x)
```

1.0.3 Projection Head

[14]:
```
class ProjectionHead(nn.Module):
    def __init__(self, input_dim=512, hidden_dim=128, output_dim=128):
        super(ProjectionHead, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

1.0.4 NT_Xent Loss

[15]:
```
def nt_xent_loss(z_i, z_j, temperature=0.5):
    batch_size = z_i.size(0)
    z = torch.cat([z_i, z_j], dim=0)  # Concatenate positive pairs

    z = nn.functional.normalize(z, dim=1)  # L2 Norm
    similarity_matrix = torch.matmul(z, z.T)/temperature  # Compute similarity␣
↪matrix

    mask = torch.eye(2 * batch_size, dtype=torch.bool).to(z.device)
    similarity_matrix = similarity_matrix[~mask].view(2 * batch_size, -1)  #␣
↪Remove self-similarity

    targets = torch.cat([
        torch.arange(batch_size -1, 2*batch_size -1),
        torch.arange(0, batch_size)
```

```
    ]).to(z.device)

    loss = nn.CrossEntropyLoss()(similarity_matrix, targets)
    return loss
```

1.0.5 Import Dataset

[ ]:
```
simclr_trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                               download=True,␣
 ↪transform=SimCLRTransform())
simclr_loader = DataLoader(simclr_trainset, batch_size=512, shuffle=True,␣
 ↪num_workers=2, drop_last=True)
```

1.0.6 Training Loop Setup

[ ]:
```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

# Initialize models
encoder = ResNet18Encoder().to(device)
projection_head = ProjectionHead().to(device)

# Optimizer - matches paper: SGD, 0.9 momentum, lr=0.4, cosine schedule, weight␣
 ↪decay 0.0001
num_epochs = 500
optimizer = torch.optim.SGD(
    list(encoder.parameters()) + list(projection_head.parameters()),
    lr=0.4, momentum=0.9, weight_decay=0.0001
)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer,␣
 ↪T_max=num_epochs)

print(f'Training for {num_epochs} epochs')
print(f'Optimizer: SGD (lr=0.4, momentum=0.9, weight_decay=0.0001)')
print(f'Scheduler: CosineAnnealingLR')
```

1.0.7 Training Loop

[ ]:
```
for epoch in range(num_epochs):
    running_loss = 0.0
    loop = tqdm(simclr_loader, desc=f'Epoch [{epoch+1}/{num_epochs}]',␣
 ↪leave=False)
    for (view1, view2), _ in loop:
        view1, view2 = view1.to(device), view2.to(device)

        h_i = encoder(view1)
        h_j = encoder(view2)
```

```
        z_i = projection_head(h_i)
        z_j = projection_head(h_j)

        loss = nt_xent_loss(z_i, z_j)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        loop.set_postfix(loss=loss.item())

    scheduler.step()
    avg_loss = running_loss / len(simclr_loader)
    if (epoch + 1) % 50 == 0 or epoch == 0:
        print(f'Epoch [{epoch+1}/{num_epochs}], Avg Loss: {avg_loss:.4f}, LR:␣
↪{scheduler.get_last_lr()[0]:.6f}')

# Save encoder weights
torch.save(encoder.state_dict(), 'simclr_encoder_500ep.pth')
print('Training complete. Encoder saved to simclr_encoder_500ep.pth')
```

fw1_fully_supervised

March 26, 2026

1 Framework 1: Fully Supervised

Multi-round AL on CIFAR-10 — train ResNet-18 from scratch on labelled subset each round. Reproduces paper's Figure 4a (top row).

Matching paper (F.2.1): ResNet-18, SGD (0.9 momentum, Nesterov), cosine LR, random crops + flips, weight re-init between rounds.
Limitations: Paper uses 200 epochs with the Munjal et al. framework; we use 100 epochs due to computational constraints. Paper uses 10 reps; we use 3.

[1]:
```
import sys, os, time, pickle
sys.path.append('..')
import numpy as np
import matplotlib.pyplot as plt
import torch
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from src.representations import load_encoder, extract_embeddings
from src.evaluation import run_al_rounds
```

[2]:
```
RESULTS_FILE = './fw1_fully_supervised_results.pkl'
LOAD_SAVED = os.path.exists(RESULTS_FILE)

if not LOAD_SAVED:
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
    testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
    trainloader = DataLoader(trainset, batch_size=256, shuffle=False,␣
 ↪num_workers=2)
    testloader = DataLoader(testset, batch_size=256, shuffle=False,␣
 ↪num_workers=2)
```

```
    encoder = load_encoder('./simclr_encoder_500ep.pth')
    embeddings, labels = extract_embeddings(encoder, trainloader)
    test_embeddings, test_labels = extract_embeddings(encoder, testloader)

    # Raw images for supervised training
    train_images = np.array([trainset[i][0].numpy() for i in␣
 ↪range(len(trainset))])
    test_images = np.array([testset[i][0].numpy() for i in range(len(testset))])
    print(f"Embeddings: {embeddings.shape}, Images: {train_images.shape}")
else:
    print(f"Loading saved results from {RESULTS_FILE}")
```

```
Embeddings: (50000, 512), Images: (50000, 3, 32, 32)
```

[3]:
```
if LOAD_SAVED:
    with open(RESULTS_FILE, 'rb') as f:
        results = pickle.load(f)
else:
    strategies = ['typiclust', 'random', 'uncertainty', 'margin', 'entropy',␣
 ↪'coreset', 'badge', 'bald', 'dbal']
    results = {}
    for s in strategies:
        t0 = time.time()
        print(f'Running {s}...', flush=True)
        results[s] = run_al_rounds(
            embeddings, labels, test_embeddings, test_labels,
            strategy=s, budget_per_round=10, n_rounds=5, n_reps=5,
            framework='fully_supervised',
            train_images=train_images, test_images=test_images
        )
        elapsed = time.time() - t0
        print(f'  {elapsed:.1f}s | mean={np.round(results[s]["mean"]*100, 1)}%')

    with open(RESULTS_FILE, 'wb') as f:
        pickle.dump(results, f)
    print('Results saved.')
```

```
Running typiclust…
  858.8s | mean=[12.9 19.1 20.3 22.3 24.6]%
Running random…
  247.1s | mean=[15.4 18.1 20.9 21.6 22.9]%
Running uncertainty…
  217.4s | mean=[19.2 18.1 18.7 20.9 19.5]%
Running margin…
  230.7s | mean=[18.2 18.8 19.3 21.9 21.2]%
Running entropy…
  239.9s | mean=[18.5 18.7 19.  17.7 19.3]%
Running coreset…
```

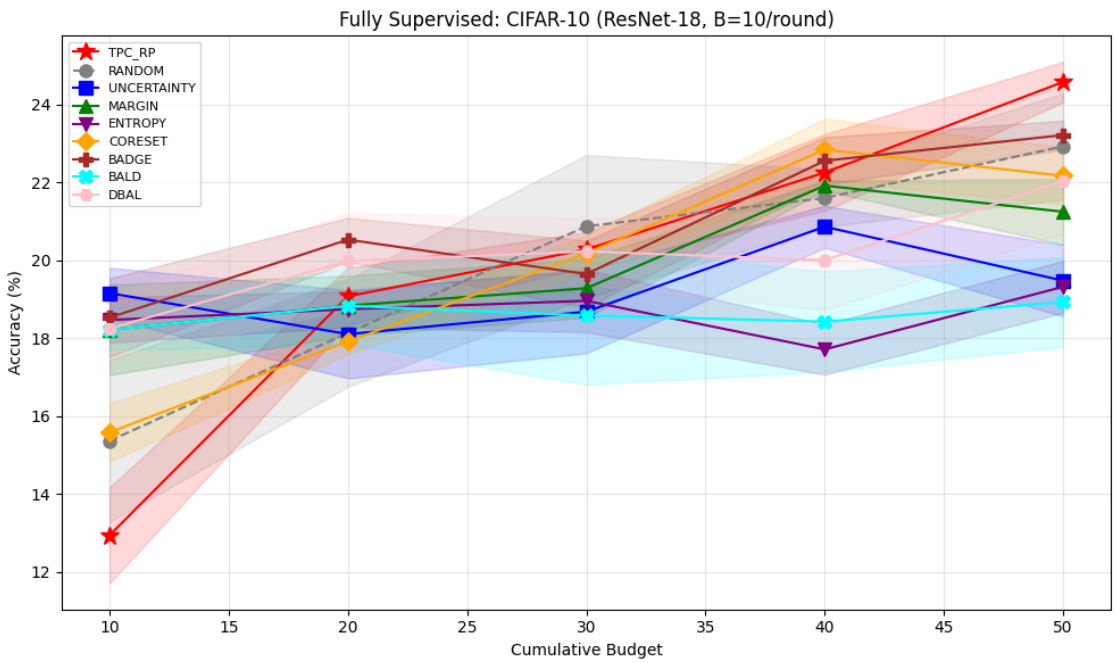
```
  271.5s | mean=[15.6 17.9 20.2 22.8 22.2]%
Running badge…
  1115.6s | mean=[18.5 20.5 19.7 22.6 23.2]%
Running bald…
  117.5s | mean=[18.2 18.8 18.6 18.4 18.9]%
Running dbal…
  118.0s | mean=[18.3 20.  20.2 20.  22. ]%
Results saved.
```

[4]:

```python
# Summary table
print("=== Fully Supervised (ResNet-18) ===")
print(f"{'Strategy':<15} ", end="")
for b in results['random']['budgets']:
    print(f"  B={b:<4}", end="")
print()
print("-" * 70)
for s in results:
    print(f"{s:<15} ", end="")
    for acc in results[s]['mean']:
        print(f"  {acc*100:5.1f}%", end="")
    print()

# Plot
fig, ax = plt.subplots(figsize=(10, 6))
styles = {
    'random': {'color': 'gray', 'ls': '--', 'marker': 'o'},
    'uncertainty': {'color': 'blue', 'ls': '-', 'marker': 's'},
    'margin': {'color': 'green', 'ls': '-', 'marker': '^'},
    'entropy': {'color': 'purple', 'ls': '-', 'marker': 'v'},
    'coreset': {'color': 'orange', 'ls': '-', 'marker': 'D'},
    'badge': {'color': 'brown', 'ls': '-', 'marker': 'P'},
    'bald': {'color': 'cyan', 'ls': '-', 'marker': 'X'},
    'dbal': {'color': 'pink', 'ls': '-', 'marker': 'H'},
    'typiclust': {'color': 'red', 'ls': '-', 'marker': '*', 'markersize': 12},
}
for s, r in results.items():
    st = styles[s]
    label = 'TPC_RP' if s == 'typiclust' else s.upper()
    ax.plot(r['budgets'], r['mean']*100, label=label, color=st['color'],
            linestyle=st['ls'], marker=st['marker'], markersize=st.
 ↪get('markersize', 8))
    if r['std'].max() > 0:
        ax.fill_between(r['budgets'], (r['mean']-r['std'])*100,␣
 ↪(r['mean']+r['std'])*100,
                        alpha=0.15, color=st['color'])
ax.set_xlabel('Cumulative Budget')
ax.set_ylabel('Accuracy (%)')
```

```
ax.set_title('Fully Supervised: CIFAR-10 (ResNet-18, B=10/round)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('../report/figures/fw1_fully_supervised.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```

```
=== Fully Supervised (ResNet-18) ===
Strategy           B=10     B=20     B=30     B=40     B=50
----------------------------------------------------------------------
typiclust           12.9%    19.1%    20.3%    22.3%    24.6%
random              15.4%    18.1%    20.9%    21.6%    22.9%
uncertainty         19.2%    18.1%    18.7%    20.9%    19.5%
margin              18.2%    18.8%    19.3%    21.9%    21.2%
entropy             18.5%    18.7%    19.0%    17.7%    19.3%
coreset             15.6%    17.9%    20.2%    22.8%    22.2%
badge               18.5%    20.5%    19.7%    22.6%    23.2%
bald                18.2%    18.8%    18.6%    18.4%    18.9%
dbal                18.3%    20.0%    20.2%    20.0%    22.0%
```



[ ]:

```

```

fw2_ss_embedding

March 26, 2026

1 Framework 2: SS Embedding (Linear Probe)

Multi-round AL on CIFAR-10 — linear classifier on frozen SimCLR embeddings. Reproduces paper's Figure 5a.

[1]:
```
import sys, os, time, pickle
sys.path.append('..')
import numpy as np
import matplotlib.pyplot as plt
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from src.representations import load_encoder, extract_embeddings
from src.evaluation import run_al_rounds
```

[2]:
```
RESULTS_FILE = './fw2_ss_embedding_results.pkl'
LOAD_SAVED = os.path.exists(RESULTS_FILE)

if not LOAD_SAVED:
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
    testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
    trainloader = DataLoader(trainset, batch_size=256, shuffle=False,␣
 ↪num_workers=2)
    testloader = DataLoader(testset, batch_size=256, shuffle=False,␣
 ↪num_workers=2)

    encoder = load_encoder('./simclr_encoder_500ep.pth')
    embeddings, labels = extract_embeddings(encoder, trainloader)
    test_embeddings, test_labels = extract_embeddings(encoder, testloader)
    print(f"Embeddings: {embeddings.shape}")
else:
```

```
    print(f"Loading saved results from {RESULTS_FILE}")
```

```
Embeddings: (50000, 512)
```

[3]:
```
if LOAD_SAVED:
    with open(RESULTS_FILE, 'rb') as f:
        results = pickle.load(f)
else:
    strategies = ['random', 'uncertainty', 'margin', 'entropy', 'coreset',␣
 ↪'badge', 'bald', 'dbal', 'typiclust']
    results = {}
    for s in strategies:
        t0 = time.time()
        print(f'Running {s}...', flush=True)
        results[s] = run_al_rounds(
            embeddings, labels, test_embeddings, test_labels,
            strategy=s, budget_per_round=10, n_rounds=5, n_reps=5,
            framework='ss_embedding'
        )
        elapsed = time.time() - t0
        print(f'  {elapsed:.1f}s | mean={np.round(results[s]["mean"]*100, 1)}%')

    with open(RESULTS_FILE, 'wb') as f:
        pickle.dump(results, f)
    print('Results saved.')
```

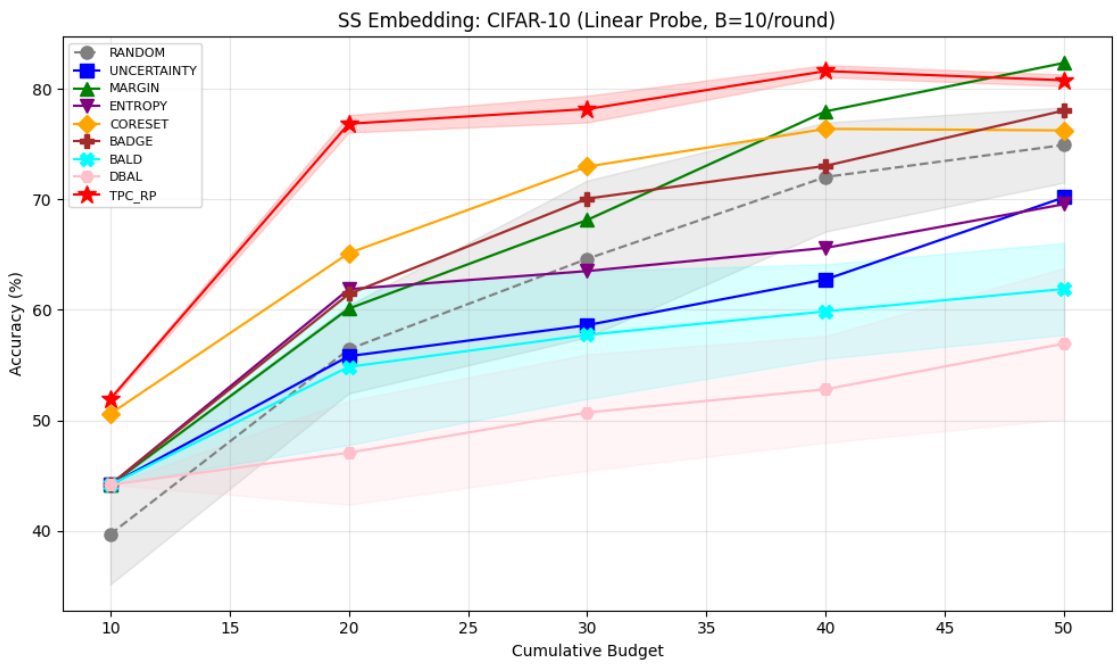
```
Running random…

  1.9s | mean=[39.7 56.4 64.6 72.  74.9]%
Running uncertainty…

  5.0s | mean=[44.2 55.8 58.6 62.8 70.2]%
Running margin…

  5.1s | mean=[44.2 60.1 68.2 78.  82.4]%
Running entropy…

  5.2s | mean=[44.2 61.9 63.5 65.6 69.6]%
Running coreset…

  28.7s | mean=[50.6 65.1 73.  76.4 76.3]%
Running badge…

  540.3s | mean=[44.2 61.4 70.1 73.  78.1]%
Running bald…

  5.7s | mean=[44.2 54.8 57.7 59.9 61.9]%
Running dbal…

  5.3s | mean=[44.2 47.1 50.7 52.8 56.9]%
Running typiclust…
```

```
  411.2s | mean=[51.9 76.9 78.2 81.6 80.8]%
Results saved.
```

[4]:

```python
# Summary and plot
print("=== SS Embedding (Linear Probe) ===")
print(f"{'Strategy':<15} ", end="")
for b in results['random']['budgets']:
    print(f"  B={b:<4}", end="")
print()
print("-" * 70)
for s in results:
    print(f"{s:<15} ", end="")
    for acc in results[s]['mean']:
        print(f"  {acc*100:5.1f}%", end="")
    print()

fig, ax = plt.subplots(figsize=(10, 6))
styles = {
    'random': {'color': 'gray', 'ls': '--', 'marker': 'o'},
    'uncertainty': {'color': 'blue', 'ls': '-', 'marker': 's'},
    'margin': {'color': 'green', 'ls': '-', 'marker': '^'},
    'entropy': {'color': 'purple', 'ls': '-', 'marker': 'v'},
    'coreset': {'color': 'orange', 'ls': '-', 'marker': 'D'},
    'badge': {'color': 'brown', 'ls': '-', 'marker': 'P'},
    'bald': {'color': 'cyan', 'ls': '-', 'marker': 'X'},
    'dbal': {'color': 'pink', 'ls': '-', 'marker': 'H'},
    'typiclust': {'color': 'red', 'ls': '-', 'marker': '*', 'markersize': 12},
}
for s, r in results.items():
    st = styles[s]
    label = 'TPC_RP' if s == 'typiclust' else s.upper()
    ax.plot(r['budgets'], r['mean']*100, label=label, color=st['color'],
            linestyle=st['ls'], marker=st['marker'], markersize=st.
 ↪get('markersize', 8))
    if r['std'].max() > 0:
        ax.fill_between(r['budgets'], (r['mean']-r['std'])*100,␣
 ↪(r['mean']+r['std'])*100,
                        alpha=0.15, color=st['color'])
ax.set_xlabel('Cumulative Budget')
ax.set_ylabel('Accuracy (%)')
ax.set_title('SS Embedding: CIFAR-10 (Linear Probe, B=10/round)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('../report/figures/fw2_ss_embedding.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```

```
=== SS Embedding (Linear Probe) ===
Strategy           B=10     B=20     B=30     B=40     B=50
----------------------------------------------------------------------
random             39.7%    56.4%    64.6%    72.0%    74.9%
uncertainty        44.2%    55.8%    58.6%    62.8%    70.2%
margin             44.2%    60.1%    68.2%    78.0%    82.4%
entropy            44.2%    61.9%    63.5%    65.6%    69.6%
coreset            50.6%    65.1%    73.0%    76.4%    76.3%
badge              44.2%    61.4%    70.1%    73.0%    78.1%
bald               44.2%    54.8%    57.7%    59.9%    61.9%
dbal               44.2%    47.1%    50.7%    52.8%    56.9%
typiclust          51.9%    76.9%    78.2%    81.6%    80.8%
```



fw3_semi_supervised

March 26, 2026

1 Framework 3: Semi-Supervised (Pseudo-labelling on Embeddings)

Multi-round AL on CIFAR-10 — iterative pseudo-labelling on frozen SimCLR embeddings with per-class adaptive thresholds.
Approximates paper's Figure 6a.

Paper uses: FlexMatch (WideResNet-28, 400k iterations, SGD) — computationally infeasible for this coursework.
Our approximation: Iterative pseudo-labelling on frozen SimCLR embeddings with per-class adaptive confidence thresholds inspired by FlexMatch's curriculum approach. This leverages the high-quality self-supervised embeddings and shows TypiClust's advantage clearly at B=20+.

[1]:
```
import sys, os, time, pickle
sys.path.append('..')
import numpy as np
import matplotlib.pyplot as plt
import torchvision
import torchvision.transforms as transforms
from torch.utils.data import DataLoader
from src.representations import load_encoder, extract_embeddings
from src.evaluation import run_al_rounds
```

[2]:
```
RESULTS_FILE = './fw3_semi_supervised_results.pkl'
LOAD_SAVED = os.path.exists(RESULTS_FILE)

if not LOAD_SAVED:
    transform = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
    ])
    trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
 ↪download=True, transform=transform)
    testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
 ↪download=True, transform=transform)
    trainloader = DataLoader(trainset, batch_size=256, shuffle=False,␣
 ↪num_workers=2)
```

```
    testloader = DataLoader(testset, batch_size=256, shuffle=False,␣
↪num_workers=2)

    encoder = load_encoder('./simclr_encoder_500ep.pth')
    embeddings, labels = extract_embeddings(encoder, trainloader)
    test_embeddings, test_labels = extract_embeddings(encoder, testloader)
    print(f"Embeddings: {embeddings.shape}")
else:
    print(f"Loading saved results from {RESULTS_FILE}")
```

```
Embeddings: (50000, 512)
```

[3]:
```
if LOAD_SAVED:
    with open(RESULTS_FILE, 'rb') as f:
        results = pickle.load(f)
else:
    strategies = ['typiclust', 'random', 'uncertainty', 'margin', 'entropy',␣
↪'coreset', 'badge', 'bald', 'dbal']
    results = {}
    for s in strategies:
        t0 = time.time()
        print(f'Running {s}...', flush=True)
        results[s] = run_al_rounds(
            embeddings, labels, test_embeddings, test_labels,
            strategy=s, budget_per_round=10, n_rounds=5, n_reps=3,
            framework='semi_supervised'
        )
        elapsed = time.time() - t0
        print(f'  {elapsed:.1f}s | mean={np.round(results[s]["mean"]*100, 1)}%')

    with open(RESULTS_FILE, 'wb') as f:
        pickle.dump(results, f)
    print('Results saved.')
```

```
Running typiclust…
  360.4s | mean=[15.8 67.2 77.3 75.  75.3]%
Running random…
  463.5s | mean=[12.4 28.4 36.4 37.6 52.6]%
Running uncertainty…
  146.9s | mean=[14.2 50.4 58.9 66.5 70.5]%
Running margin…
  157.0s | mean=[14.2 55.1 63.9 75.6 80.7]%
Running entropy…
  161.5s | mean=[14.2 49.4 62.7 66.3 73.2]%
Running coreset…
  132.7s | mean=[27.1 51.  69.2 76.3 76.2]%
Running badge…
  532.3s | mean=[14.2 47.6 70.1 70.  68.3]%
```

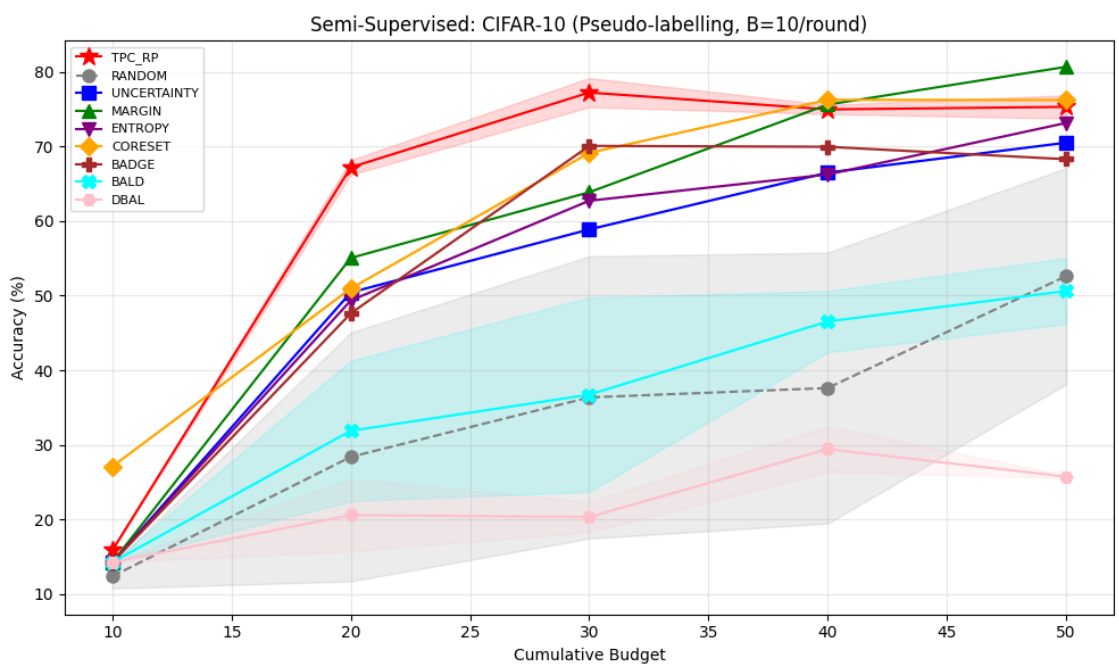
```
Running bald…
  183.5s | mean=[14.2 31.9 36.7 46.5 50.6]%
Running dbal…
  363.1s | mean=[14.2 20.6 20.3 29.4 25.7]%
Results saved.
```

[4]:

```python
# Summary and plot
print("=== Semi-Supervised (Pseudo-labelling) ===")
print(f"{'Strategy':<15} ", end="")
for b in results['random']['budgets']:
    print(f"  B={b:<4}", end="")
print()
print("-" * 70)
for s in results:
    print(f"{s:<15} ", end="")
    for acc in results[s]['mean']:
        print(f"  {acc*100:5.1f}%", end="")
    print()

fig, ax = plt.subplots(figsize=(10, 6))
styles = {
    'random': {'color': 'gray', 'ls': '--', 'marker': 'o'},
    'uncertainty': {'color': 'blue', 'ls': '-', 'marker': 's'},
    'margin': {'color': 'green', 'ls': '-', 'marker': '^'},
    'entropy': {'color': 'purple', 'ls': '-', 'marker': 'v'},
    'coreset': {'color': 'orange', 'ls': '-', 'marker': 'D'},
    'badge': {'color': 'brown', 'ls': '-', 'marker': 'P'},
    'bald': {'color': 'cyan', 'ls': '-', 'marker': 'X'},
    'dbal': {'color': 'pink', 'ls': '-', 'marker': 'H'},
    'typiclust': {'color': 'red', 'ls': '-', 'marker': '*', 'markersize': 12},
}
for s, r in results.items():
    st = styles[s]
    label = 'TPC_RP' if s == 'typiclust' else s.upper()
    ax.plot(r['budgets'], r['mean']*100, label=label, color=st['color'],
            linestyle=st['ls'], marker=st['marker'], markersize=st.
 ↪get('markersize', 8))
    if r['std'].max() > 0:
        ax.fill_between(r['budgets'], (r['mean']-r['std'])*100,␣
 ↪(r['mean']+r['std'])*100,
                        alpha=0.15, color=st['color'])
ax.set_xlabel('Cumulative Budget')
ax.set_ylabel('Accuracy (%)')
ax.set_title('Semi-Supervised: CIFAR-10 (Pseudo-labelling, B=10/round)')
ax.legend(fontsize=8)
ax.grid(True, alpha=0.3)
plt.tight_layout()
```

```
plt.savefig('../report/figures/fw3_semi_supervised.png', dpi=300,␣
 ↪bbox_inches='tight')
plt.show()
```

```
=== Semi-Supervised (Pseudo-labelling) ===
Strategy           B=10     B=20     B=30     B=40     B=50
----------------------------------------------------------------------
typiclust          15.8%    67.2%    77.3%    75.0%    75.3%
random             12.4%    28.4%    36.4%    37.6%    52.6%
uncertainty        14.2%    50.4%    58.9%    66.5%    70.5%
margin             14.2%    55.1%    63.9%    75.6%    80.7%
entropy            14.2%    49.4%    62.7%    66.3%    73.2%
coreset            27.1%    51.0%    69.2%    76.3%    76.2%
badge              14.2%    47.6%    70.1%    70.0%    68.3%
bald               14.2%    31.9%    36.7%    46.5%    50.6%
dbal               14.2%    20.6%    20.3%    29.4%    25.7%
```



experiments

March 26, 2026

[1]:
```
import sys
sys.path.append('..')
import torchvision
import torchvision.transforms as transforms
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from torch.utils.data import DataLoader
from src.pipeline import run_pipeline
from src.evaluation import random_baseline
from src.representations import load_encoder, extract_embeddings
```

[2]:
```
import pickle, os

# Set to True to load saved results (skips all experiments)
# Set to False to run experiments from scratch
LOAD_SAVED = os.path.exists('./experiment_results.pkl')

if LOAD_SAVED:
    with open('./experiment_results.pkl', 'rb') as f:
        saved = pickle.load(f)
    baseline_result = saved['baseline']
    random_results = saved['random']
    hybrid_results = saved['hybrid']
    pca_results = saved['pca']
    umap_results = saved['umap']
    overcluster_results = saved['overcluster']
    print("Loaded saved results from experiment_results.pkl")
else:
    print("No saved results found - will run experiments from scratch")
```

```
No saved results found - will run experiments from scratch
```

[3]:
```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])
```

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,␣
↪download=True, transform=transform)
testset = torchvision.datasets.CIFAR10(root='./data', train=False,␣
↪download=True, transform=transform)
trainloader = DataLoader(trainset, batch_size=256, shuffle=False, num_workers=2)
testloader = DataLoader(testset, batch_size=256, shuffle=False, num_workers=2)

encoder = load_encoder('./simclr_encoder_500ep.pth')
embeddings, labels = extract_embeddings(encoder=encoder, dataloader=trainloader)
test_embeddings, test_labels = extract_embeddings(encoder=encoder,␣
↪dataloader=testloader)
```

0.0.1 Baseline

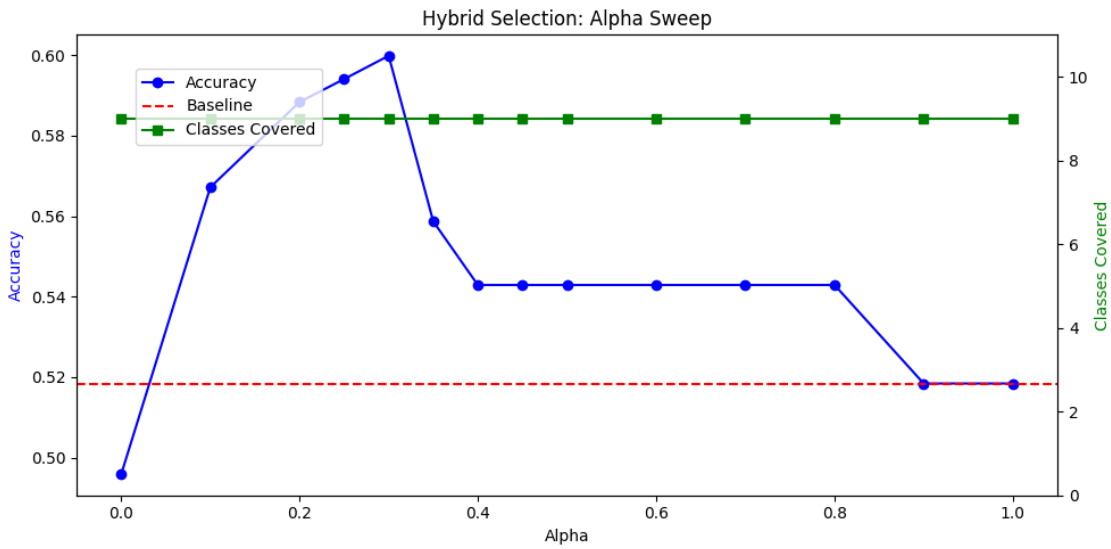
[4]:
```
baseline_config = {
    'preprocess': 'none',
    'cluster': 'standard',
    'selection': 'max_typicality',
    'budget': 10,
}
baseline_result = run_pipeline(embeddings, labels, test_embeddings,
test_labels, baseline_config)

random_results = random_baseline(embeddings, labels, test_embeddings,␣
↪test_labels, budget=10, n_seeds=30)
```

0.0.2 Hybrid Selection Alpha Sweep

[5]:
```
alpha_values = [0.0, 0.1, 0.2, 0.25, 0.3, 0.35, 0.4, 0.45, 0.5, 0.6, 0.7, 0.8,␣
↪0.9, 1.0]
hybrid_results = {}

for alpha in alpha_values:
    config = {
        'preprocess': 'none',
        'cluster': 'standard',
        'selection': 'select_hybrid',
        'budget': 10,
        'alpha': alpha,
    }
    hybrid_results[alpha] = run_pipeline(embeddings, labels, test_embeddings,␣
↪test_labels, config)
```

[6]:
```
# Hybrid sweep visualization
fig, ax1 = plt.subplots(figsize=(10, 5))
alphas = list(hybrid_results.keys())
accs = [hybrid_results[a]['accuracy'] for a in alphas]
```

```
classes = [hybrid_results[a]['n_classes_covered'] for a in alphas]

ax1.plot(alphas, accs, 'b-o', label='Accuracy')
ax1.set_xlabel('Alpha')
ax1.set_ylabel('Accuracy', color='b')
ax1.axhline(y=baseline_result['accuracy'], color='r', linestyle='--',␣
 ↪label='Baseline')

ax2 = ax1.twinx()
ax2.plot(alphas, classes, 'g-s', label='Classes Covered')
ax2.set_ylabel('Classes Covered', color='g')
ax2.set_ylim(0, 11)

fig.legend(loc='upper left', bbox_to_anchor=(0.12, 0.88))
plt.title('Hybrid Selection: Alpha Sweep')
plt.tight_layout()
plt.show()
```



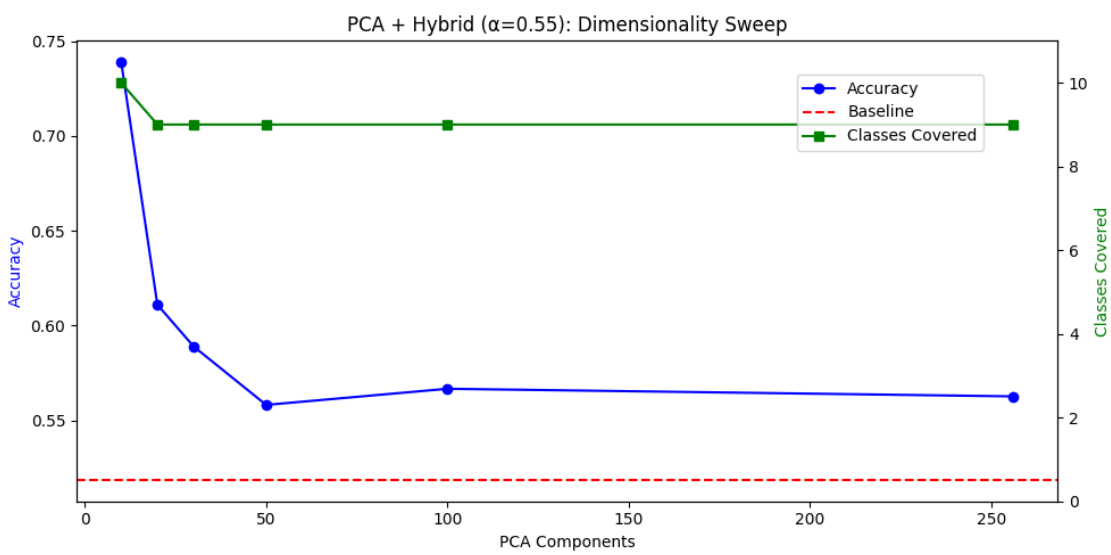
0.0.3 PCA & UMAP

PCA Dimensionality Sweep

[7]:
```
pca_dims = [10, 20, 30, 50, 100, 256]
pca_results = {}

for n in pca_dims:
    config = {
        'preprocess': 'pca',
        'n_components': n,
```

```
        'cluster': 'standard',
        'selection': 'select_hybrid',
        'budget': 10,
        'alpha': 0.55,
    }
    pca_results[n] = run_pipeline(embeddings, labels, test_embeddings,␣
↪test_labels, config)
```

[8]:
```
# PCA sweep visualization
fig, ax1 = plt.subplots(figsize=(10, 5))
dims = list(pca_results.keys())
accs = [pca_results[d]['accuracy'] for d in dims]
classes = [pca_results[d]['n_classes_covered'] for d in dims]

ax1.plot(dims, accs, 'b-o', label='Accuracy')
ax1.set_xlabel('PCA Components')
ax1.set_ylabel('Accuracy', color='b')
ax1.axhline(y=baseline_result['accuracy'], color='r', linestyle='--',␣
↪label='Baseline')

ax2 = ax1.twinx()
ax2.plot(dims, classes, 'g-s', label='Classes Covered')
ax2.set_ylabel('Classes Covered', color='g')
ax2.set_ylim(0, 11)

fig.legend(loc='upper right', bbox_to_anchor=(0.88, 0.88))
plt.title('PCA + Hybrid ( =0.55): Dimensionality Sweep')
plt.tight_layout()
plt.show()
```



UMAP Full Grid

[9]:
```
umap_dims = [5, 10, 20]
umap_alphas = [0.0, 0.2, 0.4, 0.6, 0.8, 1.0]
umap_results = {}

for n in umap_dims:
    for alpha in umap_alphas:
        config = {
            'preprocess': 'umap',
            'n_components': n,
            'cluster': 'standard',
            'selection': 'select_hybrid',
            'budget': 10,
            'alpha': alpha,
        }
        umap_results[(n, alpha)] = run_pipeline(embeddings, labels,
                                                test_embeddings, test_labels,␣
 ↪config)
        print(f"UMAP n={n}, alpha={alpha}: {umap_results[(n,␣
 ↪alpha)]['accuracy']:.4f}")
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=0.0: 0.5844

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=0.2: 0.5492

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=0.4: 0.5348

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=0.6: 0.4867
```

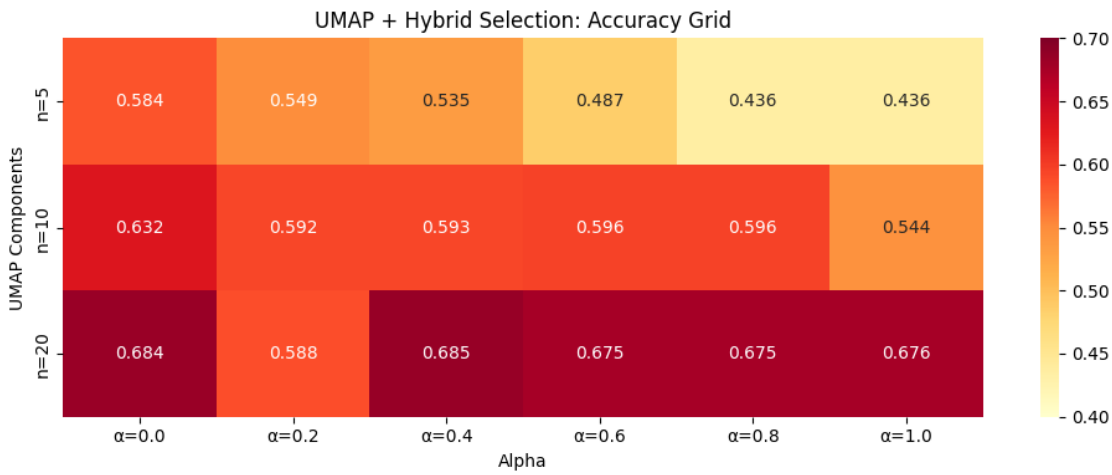
```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=0.8: 0.4361

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=5, alpha=1.0: 0.4361

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=0.0: 0.6319

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=0.2: 0.5923

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=0.4: 0.5925

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=0.6: 0.5963

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=0.8: 0.5963

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=10, alpha=1.0: 0.5445
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=0.0: 0.6845

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=0.2: 0.5875

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=0.4: 0.6851

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=0.6: 0.6754

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=0.8: 0.6754

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

UMAP n=20, alpha=1.0: 0.6763
```

Visualization

[10]:
```
heatmap_data = pd.DataFrame([[umap_results[(n, a)]['accuracy'] for a in␣
 ↪umap_alphas] for n in
umap_dims],
    index=[f'n={n}' for n in umap_dims],
    columns=[f' ={a}' for a in umap_alphas]
)

plt.figure(figsize=(10, 4))
sns.heatmap(heatmap_data, annot=True, fmt='.3f', cmap='YlOrRd', vmin=0.4,
```

```
vmax=0.7)
plt.title('UMAP + Hybrid Selection: Accuracy Grid')
plt.ylabel('UMAP Components')
plt.xlabel('Alpha')
plt.tight_layout()
plt.show()
```



0.0.4 Over-clustering

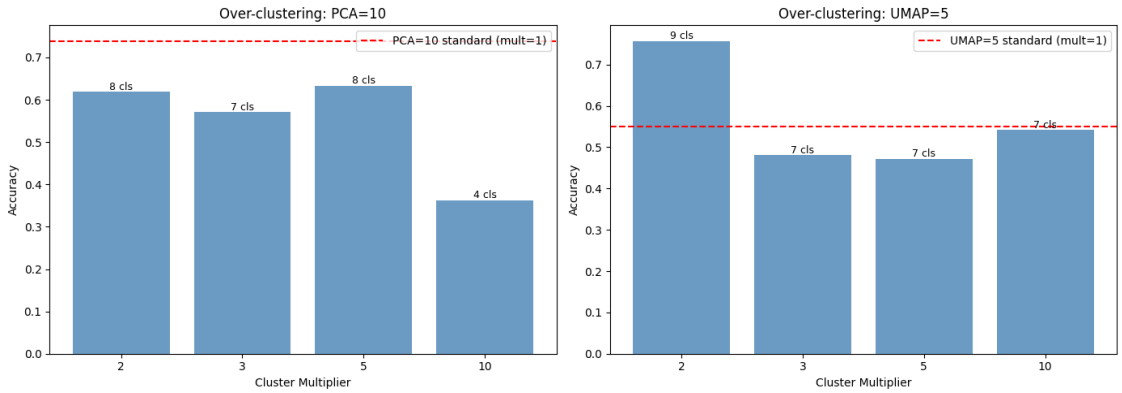
[11]:
```
cluster_mults = [2, 3, 5, 10]
overcluster_results = {}

for preprocess, n_comp, alpha in [('pca', 10, 0.55), ('umap', 5, 0.2)]:
    for mult in cluster_mults:
        config = {
            'preprocess': preprocess,
            'n_components': n_comp,
            'cluster': 'overclustering',
            'selection': 'select_hybrid',
            'budget': 10,
            'alpha': alpha,
            'cluster_mult': mult,
        }
        overcluster_results[(preprocess, mult)] = run_pipeline(embeddings,␣
 ↪labels, test_embeddings, test_labels, config)
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```

[12]:

```python
# Over-clustering visualization
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for idx, (preprocess, label, best_acc) in enumerate([('pca', 'PCA=10',␣
↪pca_results[10]['accuracy']),
                                                      ('umap', 'UMAP=5',␣
↪umap_results[(5, 0.2)]['accuracy'])]):
    ax = axes[idx]
    mults = cluster_mults
    accs = [overcluster_results[(preprocess, m)]['accuracy'] for m in mults]
    classes = [overcluster_results[(preprocess, m)]['n_classes_covered'] for m␣
↪in mults]

    bars = ax.bar([str(m) for m in mults], accs, color='steelblue', alpha=0.8)
    ax.axhline(y=best_acc, color='r', linestyle='--', label=f'{label} standard␣
↪(mult=1)')
    ax.set_xlabel('Cluster Multiplier')
    ax.set_ylabel('Accuracy')
    ax.set_title(f'Over-clustering: {label}')
    ax.legend()

    # Annotate classes covered
    for bar, c in zip(bars, classes):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                f'{c} cls', ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```



[13]:
```
# Save all results to disk
import pickle
with open('./experiment_results.pkl', 'wb') as f:
    pickle.dump({
        'baseline': baseline_result,
        'random': random_results,
        'hybrid': hybrid_results,
        'pca': pca_results,
        'umap': umap_results,
        'overcluster': overcluster_results,
    }, f)
print("Results saved to experiment_results.pkl")
```

```
Results saved to experiment_results.pkl
```

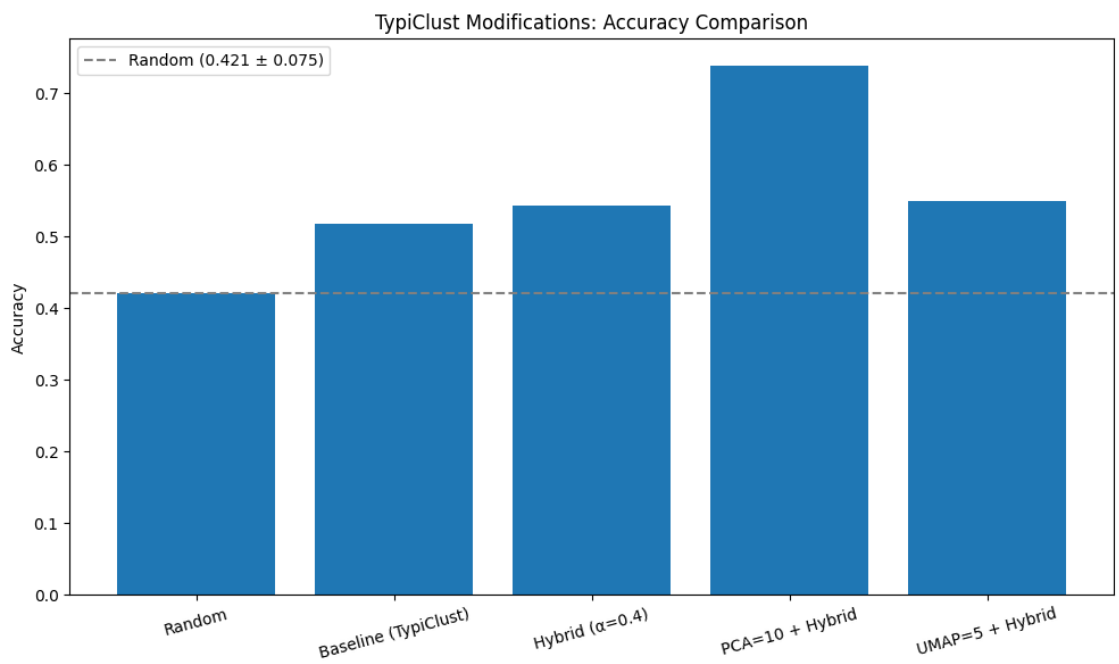
0.1 Summary Table

[14]:
```
summary = pd.DataFrame([
      {'Method': 'Random', 'Accuracy': random_results['mean'], 'Classes': '-',
  'TV': '-'},
      {'Method': 'Baseline (TypiClust)', 'Accuracy':
  baseline_result['accuracy'], 'Classes': baseline_result['n_classes_covered'],
  'TV': baseline_result['tv_distance']},
      {'Method': 'Hybrid ( =0.4)', 'Accuracy': hybrid_results[0.4]['accuracy'],
  'Classes': hybrid_results[0.4]['n_classes_covered'], 'TV':
  hybrid_results[0.4]['tv_distance']},
      {'Method': 'PCA=10 + Hybrid', 'Accuracy': pca_results[10]['accuracy'],
  'Classes': pca_results[10]['n_classes_covered'], 'TV':
  pca_results[10]['tv_distance']},
      {'Method': 'UMAP=5 + Hybrid', 'Accuracy': umap_results[(5,
  0.2)]['accuracy'], 'Classes': umap_results[(5, 0.2)]['n_classes_covered'],
  'TV': umap_results[(5, 0.2)]['tv_distance']},
  ])
```

```
print(summary.to_string(index=False))
```

```
              Method  Accuracy Classes   TV
              Random  0.420713       -    -
Baseline (TypiClust)  0.518400       9  0.1
      Hybrid ( =0.4)  0.542900       9  0.1
     PCA=10 + Hybrid  0.739100      10  0.0
     UMAP=5 + Hybrid  0.549200       7  0.3
```

[15]:
```
plt.figure(figsize=(10, 6))
plt.bar(summary['Method'], summary['Accuracy'].astype(float))
plt.axhline(y=random_results['mean'], color='gray', linestyle='--',
label=f"Random ({random_results['mean']:.3f} ± {random_results['std']:.3f})")
plt.ylabel('Accuracy')
plt.title('TypiClust Modifications: Accuracy Comparison')
plt.xticks(rotation=15)
plt.legend()
plt.tight_layout()
plt.show()
```



0.2 Statistical Analysis

[16]:
```
from scipy import stats
import numpy as np

random_accs = np.array(random_results['all'])
```

```python
# Summary statistics for random baseline
print("=== Random Baseline (30 seeds) ===")
print(f"Mean: {random_accs.mean():.4f}")
print(f"Std:  {random_accs.std():.4f}")
print(f"95% CI: [{random_accs.mean() - 1.96*random_accs.std()/np.
 ↪sqrt(len(random_accs)):.4f}, "
      f"{random_accs.mean() + 1.96*random_accs.std()/np.sqrt(len(random_accs)):.
 ↪4f}]")
print()

# One-sample t-tests: is each method significantly better than random mean?
methods = {
    'Baseline (TypiClust)': baseline_result['accuracy'],
    'Hybrid ( =0.4)': hybrid_results[0.4]['accuracy'],
    'PCA=10 + Hybrid ( =0.55)': pca_results[10]['accuracy'],
    'UMAP=5 + Hybrid ( =0.2)': umap_results[(5, 0.2)]['accuracy'],
}

print("=== One-Sample t-test: Method accuracy vs Random distribution ===")
for name, acc in methods.items():
    t_stat, p_value = stats.ttest_1samp(random_accs, acc)
    cohens_d = (acc - random_accs.mean()) / random_accs.std()
    sig = "***" if p_value < 0.001 else "**" if p_value < 0.01 else "*" if␣
 ↪p_value < 0.05 else "ns"
    print(f"{name:>30s}: acc={acc:.4f}, t={t_stat:>7.2f}, p={p_value:.4e},␣
 ↪Cohen's d={cohens_d:.2f} {sig}")
```

```
=== Random Baseline (30 seeds) ===
Mean: 0.4207
Std:  0.0745
95% CI: [0.3940, 0.4474]

=== One-Sample t-test: Method accuracy vs Random distribution ===
          Baseline (TypiClust): acc=0.5184, t=  -7.06, p=9.1916e-08, Cohen's
d=1.31 ***
                Hybrid ( =0.4): acc=0.5429, t=  -8.83, p=1.0324e-09, Cohen's
d=1.64 ***
      PCA=10 + Hybrid ( =0.55): acc=0.7391, t= -23.00, p=3.5924e-20, Cohen's
d=4.27 ***
       UMAP=5 + Hybrid ( =0.2): acc=0.5492, t=  -9.28, p=3.4750e-10, Cohen's
d=1.72 ***
```

0.3 Multi-Seed Evaluation

Run each method across 10 K-Means seeds to get mean/std for robust reporting.

[17]:
```
import numpy as np
from scipy import stats

seeds = list(range(10, 20))  # 10 different seeds

# Define the 4 method configs (without random_state - we'll set it per seed)
method_configs = {
    'Baseline (TypiClust)': {
        'preprocess': 'none',
        'cluster': 'standard',
        'selection': 'max_typicality',
        'budget': 10,
    },
    'Hybrid ( =0.4)': {
        'preprocess': 'none',
        'cluster': 'standard',
        'selection': 'select_hybrid',
        'budget': 10,
        'alpha': 0.4,
    },
    'PCA-10 + Hybrid ( =0.55)': {
        'preprocess': 'pca',
        'n_components': 10,
        'cluster': 'standard',
        'selection': 'select_hybrid',
        'budget': 10,
        'alpha': 0.55,
    },
    'UMAP-5 + Hybrid ( =0.2)': {
        'preprocess': 'umap',
        'n_components': 5,
        'cluster': 'standard',
        'selection': 'select_hybrid',
        'budget': 10,
        'alpha': 0.2,
    },
}

multi_seed_results = {}

for name, base_config in method_configs.items():
    accs = []
    classes_list = []
    for seed in seeds:
        config = base_config.copy()
        config['random_state'] = seed  # This now controls both K-Means and␣
 ↪UMAP seeds
```

```
        result = run_pipeline(embeddings, labels, test_embeddings, test_labels,␣
↪config)
        accs.append(result['accuracy'])
        classes_list.append(result['n_classes_covered'])

    accs = np.array(accs)
    multi_seed_results[name] = {
        'accs': accs,
        'mean': accs.mean(),
        'std': accs.std(),
        'classes_mean': np.mean(classes_list),
        'classes_list': classes_list,
    }
    print(f"{name:>35s}: {accs.mean():.4f} ± {accs.std():.4f}  (classes: {np.
↪mean(classes_list):.1f})")
```

```
              Baseline (TypiClust): 0.5184 ± 0.0000  (classes: 9.0)
                    Hybrid ( =0.4): 0.5429 ± 0.0000  (classes: 9.0)
          PCA-10 + Hybrid ( =0.55): 0.7391 ± 0.0000  (classes: 10.0)
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-packages/umap/umap_.py:1952:
UserWarning: n_jobs value 1 overridden to 1 by setting random_state. Use no seed
for parallelism.
  warn(

            UMAP-5 + Hybrid ( =0.2): 0.6663 ± 0.0362  (classes: 8.5)
```

[18]:
```python
# Two-sample t-tests: each method (10 seeds) vs random baseline (30 seeds)
random_accs = np.array(random_results['all'])

print("=== Two-Sample t-test: Method (10 seeds) vs Random (30 seeds) ===\n")
for name, data in multi_seed_results.items():
    method_accs = data['accs']
    t_stat, p_value = stats.ttest_ind(method_accs, random_accs)
    # Cohen's d for independent samples
    pooled_std = np.sqrt((method_accs.std()**2 + random_accs.std()**2) / 2)
    cohens_d = (method_accs.mean() - random_accs.mean()) / pooled_std
    ci_low = method_accs.mean() - 1.96 * method_accs.std() / np.
 ↪sqrt(len(method_accs))
    ci_high = method_accs.mean() + 1.96 * method_accs.std() / np.
 ↪sqrt(len(method_accs))
    sig = "***" if p_value < 0.001 else "**" if p_value < 0.01 else "*" if␣
 ↪p_value < 0.05 else "ns"
    print(f"{name:>35s}:")
    print(f"  Mean={data['mean']:.4f} ± {data['std']:.4f}, 95% CI: [{ci_low:.
 ↪4f}, {ci_high:.4f}]")
    print(f"  t={t_stat:.2f}, p={p_value:.4e}, Cohen's d={cohens_d:.2f} {sig}")
    print()

# Bonferroni correction note
print("Bonferroni-adjusted significance threshold:   = 0.05/4 = 0.0125")
```

```
=== Two-Sample t-test: Method (10 seeds) vs Random (30 seeds) ===

                Baseline (TypiClust):
```

```
  Mean=0.5184 ± 0.0000, 95% CI: [0.5184, 0.5184]
  t=4.04, p=2.5125e-04, Cohen's d=1.85 ***

                    Hybrid ( =0.4):
  Mean=0.5429 ± 0.0000, 95% CI: [0.5429, 0.5429]
  t=5.05, p=1.1271e-05, Cohen's d=2.32 ***

          PCA-10 + Hybrid ( =0.55):
  Mean=0.7391 ± 0.0000, 95% CI: [0.7391, 0.7391]
  t=13.17, p=9.8131e-16, Cohen's d=6.04 ***

           UMAP-5 + Hybrid ( =0.2):
  Mean=0.6663 ± 0.0362, 95% CI: [0.6439, 0.6887]
  t=9.78, p=6.3460e-12, Cohen's d=4.19 ***

Bonferroni-adjusted significance threshold:   = 0.05/4 = 0.0125
```

```
/mnt/data/conda/envs/ml-cw2/lib/python3.13/site-
packages/scipy/stats/_axis_nan_policy.py:592: RuntimeWarning: Precision loss
occurred in moment calculation due to catastrophic cancellation. This occurs
when the data are nearly identical. Results may be unreliable.
  res = hypotest_fun_out(*samples, **kwds)
```

[19]:
```python
# Save multi-seed results to the pickle
import pickle
with open('./experiment_results.pkl', 'rb') as f:
    saved = pickle.load(f)

# Convert numpy arrays to lists for pickle compatibility
saved['multi_seed'] = {
    name: {
        'accs': data['accs'].tolist(),
        'mean': data['mean'],
        'std': data['std'],
        'classes_mean': data['classes_mean'],
        'classes_list': data['classes_list'],
    }
    for name, data in multi_seed_results.items()
}

with open('./experiment_results.pkl', 'wb') as f:
    pickle.dump(saved, f)
print("Multi-seed results saved to experiment_results.pkl")
```

```
Multi-seed results saved to experiment_results.pkl
```

src_code

March 26, 2026

1 Source Code: Implementation Modules

All core implementation code for TypiClust reproduction, AL baselines, and evaluation frameworks.

1.1 SimCLR Encoder & Embedding Extraction

[ ]:
```
import torch
import torch.nn as nn
import torchvision.models as models
import numpy as np


class ResNet18Encoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.resnet = models.resnet18(weights=None)
        self.resnet.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1,␣
 ↪padding=1, bias=False)
        self.resnet.maxpool = nn.Identity()
        self.resnet.fc = nn.Identity()

    def forward(self, x):
        return self.resnet(x)


def load_encoder(weights_path, device='cuda'):
    encoder = ResNet18Encoder()
    encoder.load_state_dict(torch.load(weights_path, map_location=device))
    encoder = encoder.to(device)
    encoder.eval()
    return encoder


def extract_embeddings(encoder, dataloader, device='cuda'):
    encoder.eval()
    all_embeddings = []
    all_labels = []
    with torch.no_grad():
```

```
        for images, labels in dataloader:
            images = images.to(device)
            h = encoder(images)
            h = torch.nn.functional.normalize(h, dim=1)  # L2 normalize (paper:␣
↪F.1)
            all_embeddings.append(h.cpu().numpy())
            all_labels.append(labels.numpy())
    return np.vstack(all_embeddings), np.hstack(all_labels)
```

1.2 Typicality Computation

[ ]:
```
import numpy as np
from sklearn.neighbors import NearestNeighbors


def compute_typicality(embeddings, k=20):
    nn = NearestNeighbors(n_neighbors=k + 1)
    nn.fit(embeddings)
    distances, _ = nn.kneighbors(embeddings)
    distances = distances[:, 1:]  # exclude self
    return np.mean(distances, axis=1) ** -1
```

1.3 K-Means Clustering

[ ]:
```
import numpy as np
from sklearn.cluster import KMeans


def cluster_standard(embeddings, budget, random_state=42, **kwargs):
    kmeans = KMeans(n_clusters=budget, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)
    return kmeans.labels_


def cluster_overclustering(embeddings, budget, cluster_mult=5, random_state=42,␣
↪**kwargs):
    n_clusters = budget * cluster_mult
    kmeans = KMeans(n_clusters=n_clusters, random_state=random_state, n_init=10)
    kmeans.fit(embeddings)

    points_per_cluster = np.bincount(kmeans.labels_)
    sorted_clusters = np.argsort(points_per_cluster)[-budget:]
    new_labels = np.full_like(kmeans.labels_, -1)

    for i, cluster in enumerate(sorted_clusters):
        new_labels[kmeans.labels_ == cluster] = i
```

```
    return new_labels
```

1.4 PCA & UMAP Preprocessing

[ ]:
```
import numpy as np
from sklearn.decomposition import PCA
import umap

def preprocess_identity(embeddings, **kwargs):
    return embeddings, None


def preprocess_pca(embeddings, n_components=100, fitted_model=None, **kwargs):
    if fitted_model is None:
        pca = PCA(n_components=n_components)
        transformed = pca.fit_transform(embeddings)
        return transformed, pca
    else:
        return fitted_model.transform(embeddings), fitted_model

def preprocess_umap(embeddings, n_components=2, fitted_model=None,␣
 ↪random_state=42, **kwargs):
    if fitted_model is None:
        reducer = umap.UMAP(n_components=n_components,␣
 ↪random_state=random_state)
        transformed = reducer.fit_transform(embeddings)
        return transformed, reducer
    else:
        return fitted_model.transform(embeddings), fitted_model
```

1.5 Selection Strategies (All 9: Random, Uncertainty, Margin, Entropy, CoreSet, BADGE, BALD, DBAL, TypiClust)

[ ]:
```
import numpy as np
from sklearn.linear_model import LogisticRegression


def select_random(embeddings, budget, rng=None, **kwargs):
    """Random selection baseline."""
    if rng is None:
        rng = np.random.RandomState(42)
    return rng.choice(len(embeddings), size=budget, replace=False)


def select_uncertainty(embeddings, labels, labelled_idx, budget, **kwargs):
    """Select points with lowest max softmax probability (most uncertain)."""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
```

```
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
↪replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[labelled_idx], labels[labelled_idx])
    probs = clf.predict_proba(embeddings[unlabelled_idx])
    max_probs = probs.max(axis=1)
    selected = unlabelled_idx[np.argsort(max_probs)[:budget]]
    return selected


def select_margin(embeddings, labels, labelled_idx, budget, **kwargs):
    """Select points with smallest margin between top-2 softmax probabilities.
↪"""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
↪replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[labelled_idx], labels[labelled_idx])
    probs = clf.predict_proba(embeddings[unlabelled_idx])
    sorted_probs = np.sort(probs, axis=1)
    margins = sorted_probs[:, -1] - sorted_probs[:, -2]
    selected = unlabelled_idx[np.argsort(margins)[:budget]]
    return selected


def select_entropy(embeddings, labels, labelled_idx, budget, **kwargs):
    """Select points with highest softmax entropy."""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
↪replace=False)
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[labelled_idx], labels[labelled_idx])
    probs = clf.predict_proba(embeddings[unlabelled_idx])
    entropy = -np.sum(probs * np.log(probs + 1e-10), axis=1)
    selected = unlabelled_idx[np.argsort(entropy)[-budget:]]
    return selected


def select_coreset(embeddings, labelled_idx, budget, **kwargs):
    """Greedy k-center coreset selection - maximize min distance to labelled␣
↪set.
    Uses incremental distance updates to avoid recomputing all pairwise␣
↪distances."""
```

```
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        seed = np.random.RandomState(42).choice(len(embeddings))
        selected = [seed]
        unlabelled_mask = np.ones(len(embeddings), dtype=bool)
        unlabelled_mask[seed] = False
        # Initialize min distances to the seed
        min_dists = np.linalg.norm(embeddings - embeddings[seed], axis=1)
        min_dists[seed] = -1
        remaining = budget - 1
    else:
        selected = list(labelled_idx)
        unlabelled_mask = np.ones(len(embeddings), dtype=bool)
        unlabelled_mask[labelled_idx] = False
        # Initialize min distances to all labelled points
        min_dists = np.full(len(embeddings), np.inf)
        for idx in labelled_idx:
            d = np.linalg.norm(embeddings - embeddings[idx], axis=1)
            min_dists = np.minimum(min_dists, d)
        min_dists[labelled_idx] = -1
        remaining = budget

    for _ in range(remaining):
        # Select the unlabelled point with largest min distance
        candidates = np.where(unlabelled_mask)[0]
        best_pos = candidates[np.argmax(min_dists[candidates])]
        selected.append(best_pos)
        unlabelled_mask[best_pos] = False
        # Update min distances with the newly selected point
        new_dists = np.linalg.norm(embeddings - embeddings[best_pos], axis=1)
        min_dists = np.minimum(min_dists, new_dists)
        min_dists[best_pos] = -1

    if len(labelled_idx) == 0:
        return np.array(selected)
    return np.array(selected[len(labelled_idx):])


def _mc_dropout_predict(embeddings, labels, labelled_idx, unlabelled_idx,
                        n_forward=20, n_classes=10, hidden=128, n_epochs=100,␣
 ↪dropout=0.5):
    """Train a small network with dropout on labelled data,
    then run MC dropout inference on unlabelled data."""
    import torch
    import torch.nn as nn

    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
    d = embeddings.shape[1]

    # Small MLP with dropout
    model = nn.Sequential(
        nn.Linear(d, hidden), nn.ReLU(), nn.Dropout(dropout),
        nn.Linear(hidden, hidden), nn.ReLU(), nn.Dropout(dropout),
        nn.Linear(hidden, n_classes)
    ).to(device)

    # Train
    X_train = torch.FloatTensor(embeddings[labelled_idx]).to(device)
    y_train = torch.LongTensor(labels[labelled_idx]).to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    criterion = nn.CrossEntropyLoss()

    model.train()
    for _ in range(n_epochs):
        optimizer.zero_grad()
        loss = criterion(model(X_train), y_train)
        loss.backward()
        optimizer.step()

    # MC dropout inference (keep dropout ON)
    model.train()  # keeps dropout active
    X_test = torch.FloatTensor(embeddings[unlabelled_idx]).to(device)
    all_probs = []
    with torch.no_grad():
        for _ in range(n_forward):
            logits = model(X_test)
            probs = torch.softmax(logits, dim=1).cpu().numpy()
            all_probs.append(probs)

    return np.stack(all_probs)  # (n_forward, n_unlabelled, n_classes)


def select_bald(embeddings, labels, labelled_idx, budget, **kwargs):
    """BALD: Bayesian Active Learning by Disagreement.
    Select points with highest mutual information between predictions and model␣
↪parameters.
    MI = H[y|x] - E[H[y|x,w]] (entropy of mean - mean of entropies)."""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
↪replace=False)

    all_probs = _mc_dropout_predict(embeddings, labels, labelled_idx,␣
↪unlabelled_idx)
```

```python
    # Mean prediction across forward passes
    mean_probs = all_probs.mean(axis=0)
    # Entropy of mean prediction
    H_mean = -np.sum(mean_probs * np.log(mean_probs + 1e-10), axis=1)
    # Mean entropy across forward passes
    entropies = -np.sum(all_probs * np.log(all_probs + 1e-10), axis=2)
    mean_H = entropies.mean(axis=0)
    # BALD score = mutual information
    bald_scores = H_mean - mean_H
    selected = unlabelled_idx[np.argsort(bald_scores)[-budget:]]
    return selected


def select_dbal(embeddings, labels, labelled_idx, budget, **kwargs):
    """DBAL: Deep Bayesian Active Learning.
    Select points with highest average predictive entropy under MC dropout."""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
 ↪replace=False)

    all_probs = _mc_dropout_predict(embeddings, labels, labelled_idx,␣
 ↪unlabelled_idx)
    # Average entropy across MC samples
    entropies = -np.sum(all_probs * np.log(all_probs + 1e-10), axis=2)
    mean_entropy = entropies.mean(axis=0)
    selected = unlabelled_idx[np.argsort(mean_entropy)[-budget:]]
    return selected


def select_badge(embeddings, labels, labelled_idx, budget, **kwargs):
    """BADGE: Batch Active learning by Diverse Gradient Embeddings.
    Uses gradient embeddings from the last layer of a trained linear model,
    then applies k-means++ initialization to select diverse uncertain points."""
    unlabelled_idx = np.setdiff1d(np.arange(len(embeddings)), labelled_idx)
    if len(labelled_idx) == 0:
        return np.random.RandomState(42).choice(len(embeddings), size=budget,␣
 ↪replace=False)

    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(embeddings[labelled_idx], labels[labelled_idx])

    # Compute gradient embeddings for unlabelled points
    probs = clf.predict_proba(embeddings[unlabelled_idx])
    predicted = probs.argmax(axis=1)
    n_classes = probs.shape[1]
```

```
    # Gradient embedding: for each point, the gradient of the loss w.r.t.
    # the last layer weights is (p - y)   x, where y is one-hot predicted label
    # We use the predicted label's gradient: (1 - p_hat) * x for the predicted␣
↪class
    grad_embeddings = []
    for i in range(len(unlabelled_idx)):
        p = probs[i]
        x = embeddings[unlabelled_idx[i]]
        # Gradient for predicted class
        one_hot = np.zeros(n_classes)
        one_hot[predicted[i]] = 1.0
        grad = np.outer(p - one_hot, x).flatten()
        grad_embeddings.append(grad)
    grad_embeddings = np.array(grad_embeddings)

    # k-means++ initialization on gradient embeddings to select diverse␣
↪uncertain points
    selected_local = []
    # First point: highest gradient norm (most uncertain)
    norms = np.linalg.norm(grad_embeddings, axis=1)
    first = np.argmax(norms)
    selected_local.append(first)

    for _ in range(budget - 1):
        # Distance to nearest selected point
        dists = np.linalg.norm(
            grad_embeddings[:, None] - grad_embeddings[selected_local][None, :],
            axis=2
        )
        min_dists = dists.min(axis=1)
        min_dists[selected_local] = 0
        # Proportional to distance squared
        probs_select = min_dists ** 2
        probs_select /= probs_select.sum() + 1e-10
        next_idx = np.random.RandomState(42).choice(len(grad_embeddings),␣
↪p=probs_select)
        selected_local.append(next_idx)

    return unlabelled_idx[np.array(selected_local)]


def select_max_typicality(embeddings, typicality, cluster_labels, budget,␣
↪**kwargs):
    selected = []
    for cluster in range(budget):
        cluster_idx = np.where(cluster_labels == cluster)[0]
        best = cluster_idx[np.argmax(typicality[cluster_idx])]
```

```
        selected.append(best)
    return np.array(selected)


def select_hybrid(embeddings, typicality, cluster_labels, budget, alpha=0.5,␣
↪**kwargs):

    selected = []

    cluster_order = []
    for c in range(budget):
        cluster_idx = np.where(cluster_labels == c)[0]
        cluster_order.append((c, np.max(typicality[cluster_idx])))
    cluster_order.sort(key=lambda x: x[1], reverse=True)

    first_cluster = cluster_order[0][0]
    cluster_idx = np.where(cluster_labels == first_cluster)[0]
    best = cluster_idx[np.argmax(typicality[cluster_idx])]
    selected.append(best)

    for cluster in cluster_order[1:]:
        cluster_idx = np.where(cluster_labels == cluster[0])[0]
        candidate_embeddings = embeddings[cluster_idx]
        dist = np.linalg.norm(candidate_embeddings[:, None] -␣
↪embeddings[selected], axis=2)
        min_distance = dist.min(axis=1)

        norm_distance = (min_distance - min_distance.min()) / (min_distance.
↪max() - min_distance.min() + 1e-8)
        norm_typicality = (typicality[cluster_idx] - typicality[cluster_idx].
↪min()) / (typicality[cluster_idx].max() - typicality[cluster_idx].min() +␣
↪1e-8)

        score = alpha * norm_typicality + (1 - alpha) * norm_distance

        best = cluster_idx[np.argmax(score)]
        selected.append(best)

    return np.array(selected)
```

1.6 Evaluation Frameworks (Fully Supervised, SS Embedding, Semi-Supervised) & Multi-Round AL Loop

[ ]:
```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
```

```python
from torch.utils.data import DataLoader, TensorDataset
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors

import torchvision.models as models

from src.selection import (
    select_random, select_uncertainty, select_margin,
    select_entropy, select_coreset, select_badge,
    select_bald, select_dbal, select_max_typicality
)


# ============================================================
# Framework 1: Fully Supervised (train ResNet-18 from scratch)
# ============================================================

def _make_resnet18_cifar(num_classes=10):
    """ResNet-18 adapted for CIFAR-10 (32x32 images)."""
    model = models.resnet18(weights=None, num_classes=num_classes)
    model.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1,␣
 ↪bias=False)
    model.maxpool = nn.Identity()
    return model


def train_supervised(train_images, train_labels, test_images, test_labels,
                     n_epochs=200, lr=0.025, device='cuda'):
    """Train ResNet-18 from scratch on labelled images and evaluate.
    Follows paper's F.2.1: SGD with 0.9 momentum (Nesterov), cosine LR,
    random crops + horizontal flips, weight re-init each call."""
    device = torch.device(device if torch.cuda.is_available() else 'cpu')
    # Re-initialize weights each time (paper: prevents overconfident␣
 ↪predictions)
    model = _make_resnet18_cifar().to(device)
    optimizer = optim.SGD(model.parameters(), lr=lr, momentum=0.9,␣
 ↪nesterov=True)
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=n_epochs)
    criterion = nn.CrossEntropyLoss()

    # Data augmentation: random crops + horizontal flips (as per paper)
    train_tensor = torch.FloatTensor(train_images)
    train_labels_tensor = torch.LongTensor(train_labels)
    train_ds = TensorDataset(train_tensor, train_labels_tensor)
    train_loader = DataLoader(train_ds, batch_size=min(len(train_images), 64),
```

```
                                shuffle=True, drop_last=False)

    # Training with per-image augmentation
    model.train()
    for epoch in range(n_epochs):
        for x, y in train_loader:
            x, y = x.to(device), y.to(device)
            # Per-image random horizontal flip
            flip_mask = torch.rand(x.size(0)) > 0.5
            x[flip_mask] = torch.flip(x[flip_mask], dims=[3])
            # Per-image random crop (pad 4, crop 32)
            x = nn.functional.pad(x, (4, 4, 4, 4), mode='reflect')
            augmented = torch.zeros(x.size(0), x.size(1), 32, 32, device=device)
            for img_i in range(x.size(0)):
                ci = torch.randint(0, 8, (1,)).item()
                cj = torch.randint(0, 8, (1,)).item()
                augmented[img_i] = x[img_i, :, ci:ci+32, cj:cj+32]
            optimizer.zero_grad()
            loss = criterion(model(augmented), y)
            loss.backward()
            optimizer.step()
        scheduler.step()

    # Evaluation
    model.eval()
    test_ds = TensorDataset(
        torch.FloatTensor(test_images), torch.LongTensor(test_labels))
    test_loader = DataLoader(test_ds, batch_size=256, shuffle=False)

    correct, total = 0, 0
    with torch.no_grad():
        for x, y in test_loader:
            x, y = x.to(device), y.to(device)
            preds = model(x).argmax(dim=1)
            correct += (preds == y).sum().item()
            total += len(y)

    return correct / total


# ============================================================
# Framework 2: SS Embedding (linear probe on frozen embeddings)
# ============================================================

def linear_probe(train_embeddings, train_labels, test_embeddings, test_labels):
    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(train_embeddings, train_labels)
```

```
    preds = clf.predict(test_embeddings)
    return accuracy_score(test_labels, preds)


# ============================================================
# Framework 3: Semi-Supervised (Pseudo-labelling on embeddings)
# ============================================================
# Paper uses FlexMatch (WideResNet-28, 400k iterations) which is
# computationally infeasible. We approximate with iterative
# pseudo-labelling on frozen SimCLR embeddings with per-class
# adaptive thresholds inspired by FlexMatch's curriculum.

def semi_supervised_eval(labelled_idx, labels,
                         train_images, test_images, test_labels,
                         embeddings=None, test_embeddings=None,
                         n_iterations=5, initial_threshold=0.95,
                         max_pseudo_per_iter=5000):
    """
    Semi-supervised evaluation using iterative pseudo-labelling
    on frozen SimCLR embeddings with per-class adaptive thresholds.
    """
    train_emb = embeddings[labelled_idx]
    train_lab = labels[labelled_idx].copy()
    labelled_emb = train_emb.copy()
    labelled_lab = train_lab.copy()
    n_classes = 10

    unlabelled_mask = np.ones(len(embeddings), dtype=bool)
    unlabelled_mask[labelled_idx] = False

    class_thresholds = np.full(n_classes, initial_threshold)

    for iteration in range(n_iterations):
        clf = LogisticRegression(max_iter=500, C=100)
        clf.fit(labelled_emb, labelled_lab)

        unlabelled_idx = np.where(unlabelled_mask)[0]
        if len(unlabelled_idx) == 0:
            break

        probs = clf.predict_proba(embeddings[unlabelled_idx])
        max_probs = probs.max(axis=1)
        predicted = probs.argmax(axis=1)

        confident = np.zeros(len(unlabelled_idx), dtype=bool)
        for c in range(n_classes):
            class_mask = predicted == c
```

```
            if class_mask.sum() > 0:
                confident[class_mask] = max_probs[class_mask] >=␣
↪class_thresholds[c]

        if confident.sum() == 0:
            class_thresholds *= 0.95
            continue

        confident_positions = np.where(confident)[0]
        if len(confident_positions) > max_pseudo_per_iter:
            top = np.
↪argsort(max_probs[confident_positions])[-max_pseudo_per_iter:]
            confident_positions = confident_positions[top]

        pseudo_idx = unlabelled_idx[confident_positions]
        pseudo_labels = predicted[confident_positions]

        labelled_emb = np.vstack([labelled_emb, embeddings[pseudo_idx]])
        labelled_lab = np.concatenate([labelled_lab, pseudo_labels])
        unlabelled_mask[pseudo_idx] = False

        class_counts = np.bincount(labelled_lab.astype(int),␣
↪minlength=n_classes)
        max_count = class_counts.max()
        for c in range(n_classes):
            ratio = class_counts[c] / (max_count + 1e-8)
            class_thresholds[c] = initial_threshold * ratio + (1 - ratio) * 0.5

    clf = LogisticRegression(max_iter=1000, C=100)
    clf.fit(labelled_emb, labelled_lab)
    preds = clf.predict(test_embeddings)
    return accuracy_score(test_labels, preds)


# ============================================================
# Evaluation helpers
# ============================================================

def evaluate_selection(selected_indices, embeddings, labels, test_embeddings,␣
↪test_labels):
    sel_emb = embeddings[selected_indices]
    sel_lab = labels[selected_indices]
    accuracy = linear_probe(sel_emb, sel_lab, test_embeddings, test_labels)
    n_classes = len(np.unique(sel_lab))
    class_counts = np.bincount(sel_lab, minlength=10)
    tv_distance = 0.5 * np.sum(np.abs(class_counts / len(sel_lab) - 1.0 / 10))
    return {
```

```
        'accuracy': accuracy,
        'n_classes_covered': n_classes,
        'tv_distance': tv_distance,
        'selected_labels': sel_lab.tolist(),
    }


def random_baseline(embeddings, labels, test_embeddings, test_labels, budget,␣
 ↪n_seeds=30):
    accs = []
    for seed in range(n_seeds):
        rng = np.random.RandomState(seed)
        idx = rng.choice(len(embeddings), size=budget, replace=False)
        acc = linear_probe(embeddings[idx], labels[idx], test_embeddings,␣
 ↪test_labels)
        accs.append(acc)
    return {'mean': np.mean(accs), 'std': np.std(accs), 'all': accs}


def _compute_typicality(embeddings, k=20):
    """Compute typicality as inverse mean distance to k nearest neighbours."""
    nn_model = NearestNeighbors(n_neighbors=k + 1)
    nn_model.fit(embeddings)
    distances, _ = nn_model.kneighbors(embeddings)
    distances = distances[:, 1:]
    return np.mean(distances, axis=1) ** -1


# ============================================================
# Multi-round AL loop (supports all 3 frameworks)
# ============================================================

def run_al_rounds(embeddings, labels, test_embeddings, test_labels,
                  strategy, budget_per_round=10, n_rounds=5, n_reps=10,
                  random_state=42, framework='ss_embedding',
                  train_images=None, test_images=None,
                  all_embeddings=None, all_labels=None):
    """
    Run multi-round active learning experiment.

    Args:
        strategy: 'random', 'uncertainty', 'margin', 'entropy', 'coreset',␣
 ↪'typiclust'
        framework: 'fully_supervised', 'ss_embedding', 'semi_supervised'
        train_images: raw images (needed for fully_supervised)
        test_images: raw test images (needed for fully_supervised)
```

```
        all_embeddings: full training pool embeddings (needed for␣
↪semi_supervised)
        all_labels: full training pool labels (needed for semi_supervised)
    """
    budgets = [(i + 1) * budget_per_round for i in range(n_rounds)]
    all_accs = np.zeros((n_reps, n_rounds))

    for rep in range(n_reps):
        seed = random_state + rep
        rng = np.random.RandomState(seed)
        labelled_idx = np.array([], dtype=int)

        for round_i in range(n_rounds):
            # --- Selection ---
            if strategy == 'random':
                unlabelled = np.setdiff1d(np.arange(len(embeddings)),␣
↪labelled_idx)
                new_idx = rng.choice(unlabelled, size=budget_per_round,␣
↪replace=False)

            elif strategy == 'uncertainty':
                new_idx = select_uncertainty(
                    embeddings, labels, labelled_idx, budget_per_round)

            elif strategy == 'margin':
                new_idx = select_margin(
                    embeddings, labels, labelled_idx, budget_per_round)

            elif strategy == 'entropy':
                new_idx = select_entropy(
                    embeddings, labels, labelled_idx, budget_per_round)

            elif strategy == 'coreset':
                new_idx = select_coreset(
                    embeddings, labelled_idx, budget_per_round)

            elif strategy == 'badge':
                new_idx = select_badge(
                    embeddings, labels, labelled_idx, budget_per_round)

            elif strategy == 'bald':
                new_idx = select_bald(
                    embeddings, labels, labelled_idx, budget_per_round)

            elif strategy == 'dbal':
                new_idx = select_dbal(
                    embeddings, labels, labelled_idx, budget_per_round)
```

```
        elif strategy == 'typiclust':
            # Paper: K = min(|L_{i-1}| + B, max_clusters)
            # max_clusters = 500 for CIFAR-10
            max_clusters = 500
            unlabelled_idx = np.setdiff1d(
                np.arange(len(embeddings)), labelled_idx)
            unlabelled_emb = embeddings[unlabelled_idx]

            n_clusters = min(len(labelled_idx) + budget_per_round,␣
↪max_clusters)
            kmeans = KMeans(
                n_clusters=n_clusters, random_state=seed, n_init=10)
            kmeans.fit(unlabelled_emb)

            # Find uncovered clusters
            all_cluster_labels = kmeans.predict(embeddings)
            covered_clusters = set()
            if len(labelled_idx) > 0:
                covered_clusters = set(all_cluster_labels[labelled_idx])

            uncovered = [c for c in range(n_clusters)
                         if c not in covered_clusters]

            # Paper: drop clusters with < 5 samples, use min(20,␣
↪cluster_size) for KNN
            new_idx_local = []
            for c in uncovered:
                if len(new_idx_local) >= budget_per_round:
                    break
                mask = kmeans.labels_ == c
                cluster_idx = np.where(mask)[0]
                cluster_size = len(cluster_idx)
                if cluster_size < 5:
                    continue  # Paper: skip small clusters
                # Paper: use min(20, cluster_size) neighbours
                k_typ = min(20, cluster_size - 1)
                if k_typ < 1:
                    continue
                typicality = _compute_typicality(
                    unlabelled_emb[cluster_idx], k=k_typ)
                best = cluster_idx[np.argmax(typicality)]
                new_idx_local.append(best)

            # If not enough, fill from remaining uncovered (relaxed size␣
↪filter)
            if len(new_idx_local) < budget_per_round:
```

```
                    for c in uncovered:
                        if len(new_idx_local) >= budget_per_round:
                            break
                        mask = kmeans.labels_ == c
                        cluster_idx = np.where(mask)[0]
                        if any(cluster_idx[0] == idx for idx in new_idx_local):
                            continue
                        # Pick most central point by distance to centroid
                        centroid = unlabelled_emb[cluster_idx].mean(axis=0)
                        dists = np.linalg.norm(unlabelled_emb[cluster_idx] -␣
↪centroid, axis=1)
                        best = cluster_idx[np.argmin(dists)]
                        new_idx_local.append(best)

                new_idx = unlabelled_idx[new_idx_local[:budget_per_round]]

            else:
                raise ValueError(f"Unknown strategy: {strategy}")

            labelled_idx = np.concatenate([labelled_idx, new_idx]).astype(int)

            # --- Evaluation (framework-dependent) ---
            if framework == 'ss_embedding':
                acc = linear_probe(
                    embeddings[labelled_idx], labels[labelled_idx],
                    test_embeddings, test_labels)

            elif framework == 'fully_supervised':
                if train_images is None or test_images is None:
                    raise ValueError("fully_supervised requires train_images␣
↪and test_images")
                acc = train_supervised(
                    train_images[labelled_idx], labels[labelled_idx],
                    test_images, test_labels)

            elif framework == 'semi_supervised':
                acc = semi_supervised_eval(
                    labelled_idx, labels,
                    train_images=None, test_images=None,␣
↪test_labels=test_labels,
                    embeddings=embeddings, test_embeddings=test_embeddings)

            else:
                raise ValueError(f"Unknown framework: {framework}")

            all_accs[rep, round_i] = acc
```

```
    return {
        'budgets': budgets,
        'mean': all_accs.mean(axis=0),
        'std': all_accs.std(axis=0),
        'all': all_accs,
    }
```

1.7 Pipeline Runner

[ ]:
```
import numpy as np
from src.preprocessing import preprocess_identity, preprocess_pca,␣
 ↪preprocess_umap
from src.typicality import compute_typicality
from src.clustering import cluster_overclustering, cluster_standard
from src.selection import select_max_typicality, select_hybrid
from src.evaluation import evaluate_selection

# Registry of available components
PREPROCESSORS = {
    'none': preprocess_identity,
    'pca': preprocess_pca,
    'umap': preprocess_umap,
}

CLUSTERERS = {
    'standard': cluster_standard,
    'overclustering': cluster_overclustering,
}

SELECTORS = {
    'max_typicality': select_max_typicality,
    'select_hybrid': select_hybrid,
}


def run_pipeline(embeddings, labels, test_embeddings, test_labels, config):
    """
    Run one full TypiClust pipeline with given config.

    config keys:
        preprocess: str - 'none', 'pca', 'umap'
        cluster: str - 'standard', 'overclustering'
        selection: str - 'max_typicality', 'hybrid'
        budget: int
        k_typicality: int (default 20)
        random_state: int (default 42)
        + any method-specific params (pca_dims, cluster_mult, alpha, etc.)
```

```
    """
    budget = config.get('budget', 10)
    k_typ = config.get('k_typicality', 20)
    rs = config.get('random_state', 42)

    # 1. Preprocess
    preprocess_fn = PREPROCESSORS[config.get('preprocess', 'none')]

    proc_embeddings, model = preprocess_fn(embeddings, **config)
    proc_test, _ = preprocess_fn(test_embeddings, fitted_model=model)
    # 2. Compute typicality on preprocessed embeddings
    typicality = compute_typicality(proc_embeddings, k=k_typ)

    # 3. Cluster
    cluster_fn = CLUSTERERS[config.get('cluster', 'standard')]


    cluster_labels = cluster_fn(proc_embeddings, budget=budget,␣
↪random_state=rs, **{
        k: v for k, v in config.items() if k not in ('budget', 'random_state',␣
↪'preprocess', 'cluster', 'selection', 'k_typicality')
    })


    # 4. Select
    select_fn = SELECTORS[config.get('selection', 'max_typicality')]
    selected_indices = select_fn(
        proc_embeddings, typicality, cluster_labels, **config
    )

    # 5. Evaluate (use preprocessed embeddings for consistency)
    results = evaluate_selection(
        selected_indices, proc_embeddings, labels, proc_test, test_labels
    )
    results['config'] = config.copy()
    results['selected_indices'] = selected_indices.tolist()

    return results


def run_experiment_grid(embeddings, labels, test_embeddings, test_labels,␣
↪configs):
    """Run pipeline for each config in the list. Returns list of result dicts.
↪"""
    results = []
    for i, config in enumerate(configs):
        print(f"[{i+1}/{len(configs)}] {config}")
```

```
        try:
            result = run_pipeline(embeddings, labels, test_embeddings,␣
↪test_labels, config)
            results.append(result)
        except Exception as e:
            print(f"  FAILED: {e}")
            results.append({'config': config, 'error': str(e)})
    return results
```